* INDEX *

1.INTRODUCTION

1.1 Project Background

The rapid growth of the information technology and employment sectors has resulted in a large number of job opportunities and an increasing volume of job descriptions published on online recruitment platforms. A job description generally contains information about the job role, required technical skills, soft skills, educational qualifications, experience, tools, technologies, and other candidate requirements. Identifying the important skills from these descriptions manually can be time-consuming and may lead to inconsistent results.

The **SkillSense – Intelligent Job Skill Analysis** project is developed to automate the analysis of job descriptions and identify the skills required for a particular job role. The system uses **Natural Language Processing (NLP)** techniques to process unstructured job-description text and extract relevant skills. The extracted information can help users understand the skill requirements of a job more efficiently.

The project focuses on converting unstructured textual information into structured and meaningful skill information. This can support applications such as job analysis, skill-gap identification, candidate profiling, recruitment assistance, and career planning.

1.2 Overview of Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, process, and analyze human language. Since textual data is generally unstructured, NLP techniques are used to transform natural-language text into a form that can be processed by machine learning and deep learning models.

NLP is widely used in applications such as text classification, sentiment analysis, machine translation, information extraction, question answering, chatbots, and named entity recognition. Common NLP preprocessing operations include **tokenization, text normalization, stop-word handling, and linguistic analysis**.

For the SkillSense project, NLP provides the foundation for understanding job-description text. Instead of treating the job description as simple text, the system analyzes the words and phrases to identify entities that represent relevant skills and technologies.

1.3 Job-Skill Extraction Concept

Job-skill extraction is the process of automatically identifying skills and competencies mentioned in a job description. These skills may include programming languages, frameworks, databases, cloud technologies, software tools, methodologies, and other professional competencies.

For example, a job description may contain the following statement:

"The candidate should have experience in Python, SQL, Machine Learning, and TensorFlow."

From this text, a skill extraction system can identify:

- Python
- SQL
- Machine Learning
- TensorFlow

Traditional keyword-based approaches can identify predefined terms, but they may have difficulty understanding skills expressed in different contexts or formats. NLP and machine learning-based approaches can provide a more intelligent method for identifying skill-related entities from unstructured text.

In this project, **Named Entity Recognition (NER)** is used as an important technique for identifying skill entities from job descriptions. A fine-tuned **DistilBERT-based NER model** can

analyze the contextual meaning of words and phrases and classify relevant portions of the text as skill-related entities.

1.4 Role of NLP in SkillSense

The SkillSense system applies NLP to transform a raw job description into useful structured information. The general workflow consists of accepting the job description as input, processing the text, identifying relevant skill entities using the trained NLP model, and presenting the extracted skills to the user.

The basic process can be represented as:

Job Description → Text Processing → NLP Model → Skill Extraction → Structured Skill Output

This approach reduces the need for manual examination of lengthy job descriptions and provides a consistent method for identifying the skills required for different job roles.

1.5 Need for the Project

The project is motivated by the following requirements:

1. **Automation:** Reduce the effort required to manually identify skills from job descriptions.
2. **Efficient Analysis:** Process large amounts of job-description text quickly.
3. **Contextual Understanding:** Use NLP models to identify skills based on their context rather than relying only on simple keyword matching.
4. **Structured Information:** Convert unstructured job descriptions into an organized list of relevant skills.
5. **Recruitment Support:** Assist in understanding the technical and professional requirements of job roles.
6. **Career Support:** Help users identify the skills expected for specific job positions.

1.6 Project Objective

The primary objective of the SkillSense project is to develop an intelligent NLP-based system that can automatically analyze job descriptions and extract relevant skills. The system aims to provide accurate and structured skill information from unstructured job-description text, thereby making job-skill analysis faster, easier, and more consistent.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Job descriptions available on recruitment platforms are generally written in unstructured natural language and contain a mixture of technical skills, soft skills, qualifications, experience requirements, tools, and other information. Extracting the relevant skills manually from these descriptions is time-consuming and may result in missed or incorrectly identified skills.

Traditional keyword-based approaches also have limitations because the same skill may appear in different forms or contexts. Therefore, there is a need for an intelligent system that can automatically process job-description text, understand its context, and identify relevant skills.

The **SkillSense – Intelligent Job Skill Analysis** project addresses this problem by developing an NLP-based system that analyzes job descriptions and automatically extracts relevant skills using a trained Named Entity Recognition (NER) model.

2.2 Business Objective

The primary business objective of SkillSense is to simplify and automate the analysis of job descriptions.

The system aims to:

- Reduce the time required to identify skills from job descriptions.
- Provide a structured representation of required job skills.
- Support recruiters in understanding skill requirements more efficiently.
- Help organizations analyze skill requirements across different job roles.
- Assist candidates in understanding the technical and professional skills expected for a particular position.
- Reduce dependency on completely manual job-description analysis.

2.3 Technical Objective

The technical objectives of the project are:

1. To develop an NLP-based application for job-description analysis.
2. To preprocess and analyze unstructured job-description text.
3. To identify relevant skill entities from the input text.
4. To implement a **Named Entity Recognition (NER)** approach for skill extraction.
5. To utilize a fine-tuned **DistilBERT-based NER model** for contextual skill identification.
6. To integrate the trained NLP model with a user-friendly web application.
7. To display the extracted skills in a structured and understandable format.
8. To evaluate the performance of the skill extraction model using standard evaluation metrics such as **Precision, Recall, and F1-Score**.

2.4 Expected Outcome

The expected outcome of the project is a functional intelligent job-skill analysis system that accepts a job description as input and automatically identifies relevant skills.

The system is expected to:

- Accept unstructured job-description text from the user.
- Process the input using NLP techniques.
- Detect skill-related entities using the trained NER model.
- Extract and organize the identified skills.
- Present the results through a web-based interface.

Provide a faster and more consistent alternative to manual skill identification.

The final system demonstrates how NLP and transformer-based models can be applied to real-world recruitment and job-market data to convert unstructured job descriptions into meaningful structured skill information.

3. PROJECT SCOPE & USE CASE

3.1 Project Scope

The scope of the **SkillSense – Intelligent Job Skill Analysis** project is to develop an NLP-based system that can analyze unstructured job descriptions and automatically extract relevant skills. The system focuses primarily on identifying technical and professional skills mentioned in job-description text using a trained Named Entity Recognition (NER) model. The project covers the complete process from receiving job-description text as input to presenting the extracted skills as structured output. It combines Natural Language Processing, transformer-based language models, and a web-based interface to provide an easy-to-use job-skill analysis solution.

The major areas included within the project scope are:

- Collection and preparation of job-description text.
- Text preprocessing and NLP-based text analysis.
- Identification of skill-related entities.
- Fine-tuning and utilization of a **DistilBERT-based NER model**.
- Automatic extraction of skills from job descriptions.
- Integration of the NLP model with a web application.
- Display of extracted skills in a structured format.
- Evaluation of the skill extraction model using **Precision, Recall, and F1-Score**.

The current scope is focused on **job-skill extraction from job descriptions**. It does not aim to replace complete recruitment processes such as candidate interviews, background verification, final hiring decisions, or employee performance evaluation.

3.2 Recruitment Use Case

SkillSense can be used as an assisting tool in the recruitment process. Recruiters often need to examine multiple job descriptions to understand the skills required for different positions. The proposed system can automate the initial skill-identification stage.

A typical recruitment use case is:

Job Description → SkillSense → NLP Processing → Skill Extraction → Structured Skill List → Recruiter Analysis

For example, when a recruiter provides a job description containing requirements such as Python, SQL, Machine Learning, TensorFlow, and communication skills, SkillSense analyzes the text and extracts the relevant skill entities.

The recruiter can then use the extracted information to:

- Understand the core skills required for the position.
- Prepare or refine job descriptions.
- Compare skill requirements across different roles.
- Support candidate skill matching.
- Identify frequently requested technologies.
- Reduce the time spent manually reviewing job descriptions.

The system therefore acts as a **decision-support and job-analysis tool**, rather than making the final recruitment decision.

3.3 Practical Applications

The SkillSense system can have several practical applications in recruitment and career-oriented environments.

3.3.1 Recruitment Assistance
Recruiters can use the system to quickly identify the important skills associated with a job role and use them during the initial recruitment analysis.

3.3.2 Candidate Skill Matching
Extracted job skills can be compared with candidate resume skills to determine the degree of alignment between a candidate and a job requirement.

3.3.3 Skill Gap Analysis
The extracted skills can help candidates identify which skills they may need to develop for a specific job role.

3.3.4 Job Description Analysis
Organizations can analyze job descriptions to identify the technical skills and competencies most frequently required for particular positions.

3.3.5 Career Planning
Students and job seekers can use extracted skill information to understand the skills expected for their target roles and plan their learning accordingly.

3.3.6 Recruitment Analytics
Aggregated skill-extraction results can potentially be used to identify trends in technology and skill requirements across different job roles and industries.

3.4 Intended Users
The major intended users of SkillSense include:

- **Recruiters and HR professionals** – for faster job-description analysis.
- **Job seekers** – for understanding required skills.
- **Students and fresh graduates** – for identifying industry-relevant skills.
- **Career counselors** – for supporting skill and career planning.
- **Organizations** – for analyzing job-role skill requirements.
-

3.5 Scope Limitations
Although SkillSense provides automated skill extraction, the quality of the output depends on the training data, model performance, and the way skills are expressed in the input job description.

The system is primarily designed for skill extraction and does not independently perform final candidate selection, interviews, salary negotiation, or hiring decisions. Therefore, the extracted results should be considered as **supporting information for human decision-making** rather than a complete automated recruitment solution.

4. TECHNOLOGY & TOOLS USED

The SkillSense – Intelligent Job Skill Analysis project is developed using Python-based technologies and Natural Language Processing libraries. These tools are used for data preparation, text preprocessing, model development, skill extraction, model evaluation, data visualization, and web application development.

4.1 Python

Python is the primary programming language used for developing the SkillSense system. It provides extensive support for Natural Language Processing, machine learning, data analysis, visualization, and web application development.

Python is used throughout the project for:

- Data preprocessing and manipulation.
- NLP operations.
- Model training and inference.
- Skill extraction.
- Model evaluation.
- Web application integration.

4.2 Pandas

Pandas is a Python library used for data manipulation and analysis. It provides DataFrame-based structures for organizing and processing datasets.

In the SkillSense project, Pandas is used for:

- Loading datasets.
- Handling job-description data.
- Cleaning and preparing data.
- Organizing training and testing data.
- Performing data analysis.

4.3 NumPy

NumPy is a Python library used for numerical computing and array-based operations. It provides efficient numerical data structures and mathematical functions.

In the project, NumPy supports numerical operations required during data processing, machine learning, and evaluation workflows.

4.4 NLTK

Natural Language Toolkit (NLTK) is a Python library used for Natural Language Processing and text processing tasks.

It is used to support operations such as:

- Tokenization.
- Text preprocessing.
- Stop-word handling.
- Basic linguistic analysis.

NLTK helps prepare textual data before further NLP processing.

4.5 spaCy

spaCy is an NLP library designed for efficient processing of natural-language text. It provides tools for tokenization, linguistic processing, and Named Entity Recognition.

In the SkillSense project, spaCy is used for NLP-related text processing and analysis during the development of the skill extraction system.

4.6 Scikit-learn

Scikit-learn is a machine learning library that provides tools for data preprocessing, model evaluation, and statistical analysis.

In the project, Scikit-learn is used for:

- Train-test data splitting.
- Data preprocessing.
- Evaluation of model performance.
- Calculation of **Precision, Recall, and F1-Score**.

4.7 Transformers

The **Transformers** library is used to implement transformer-based Natural Language Processing models.

In SkillSense, Transformers is used for the **DistilBERT-based Named Entity Recognition (NER) model**. The trained model analyzes job-description text and identifies relevant skill entities using contextual language understanding.

The transformer-based approach allows the system to understand the relationship between words and their surrounding context rather than depending only on predefined keywords.

4.8 Matplotlib

Matplotlib is a Python visualization library used to create graphs and charts.

In the SkillSense project, Matplotlib is used for visualizing model performance and analysis results. It can be used to generate graphical representations of evaluation metrics such as:

- Precision.
- Recall.
- F1-Score.
- Model performance comparisons.

These visualizations make it easier to understand and interpret the performance of the developed NLP model.

4.9 Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating statistical and analytical visualizations.

In the project, Seaborn is used for presenting data-analysis and model-evaluation results in a clear graphical form. It can be used for visualizations such as:

- Performance comparison charts.
- Distribution plots.
- Confusion matrices.
- Statistical analysis of datasets.

Seaborn improves the readability and presentation of analytical results during the model development and evaluation stages.

4.10 Streamlit

Streamlit is a Python framework used to build interactive web applications for data science and machine learning projects.

In SkillSense, Streamlit provides the user interface for the developed NLP system. Through the web application, users can:

1. Enter or paste a job description.
2. Submit the text for processing.
3. Run the trained NLP model.
4. Extract relevant skills.
5. View the extracted skills in a structured format.

Streamlit connects the trained NLP model with the user-facing application and makes the system easier to use.

4.10 Power BI

Power BI is a business intelligence and data visualization tool used to create interactive dashboards and present analytical results in an easy-to-understand format.
In the SkillSense project, Power BI is used for visualizing skill-related analysis and model results. It can help present information such as skill frequency, skill distribution, job-role requirements, and other extracted-skill insights through interactive charts and dashboards. Power BI helps convert the extracted and processed data into meaningful visual insights, making the results easier to interpret for recruiters, analysts, and other users.

4.12 Technology Summary

Technology / Tool	Purpose in SkillSense
Python	Core programming and application development
Pandas	Dataset handling and data analysis
NumPy	Numerical and array operations
NLTK	Text preprocessing and NLP operations
spaCy	NLP processing and linguistic analysis
Scikit-learn	Data processing and model evaluation
Transformers	DistilBERT-based NER model
Matplotlib	Performance and result visualization
Seaborn	Statistical and analytical visualization
Streamlit	Interactive web application interface

These technologies collectively support the development of the SkillSense system, covering the complete workflow from data preparation and NLP processing to model evaluation, visualization, and web-based skill extraction.

5. SYSTEM ARCHITECTURE & METHODOLOGY

5.1 System Architecture

The **SkillSense – Intelligent Job Skill Analysis** system follows an end-to-end NLP pipeline for extracting relevant skills from job descriptions. The system combines text preprocessing, a transformer-based NER model, post-processing, and a web-based interface.
The overall architecture is:
Job Description → Text Preprocessing → Tokenization → DistilBERT NER Model → Skill Extraction → Post-Processing → Extracted Skills → Streamlit Interface

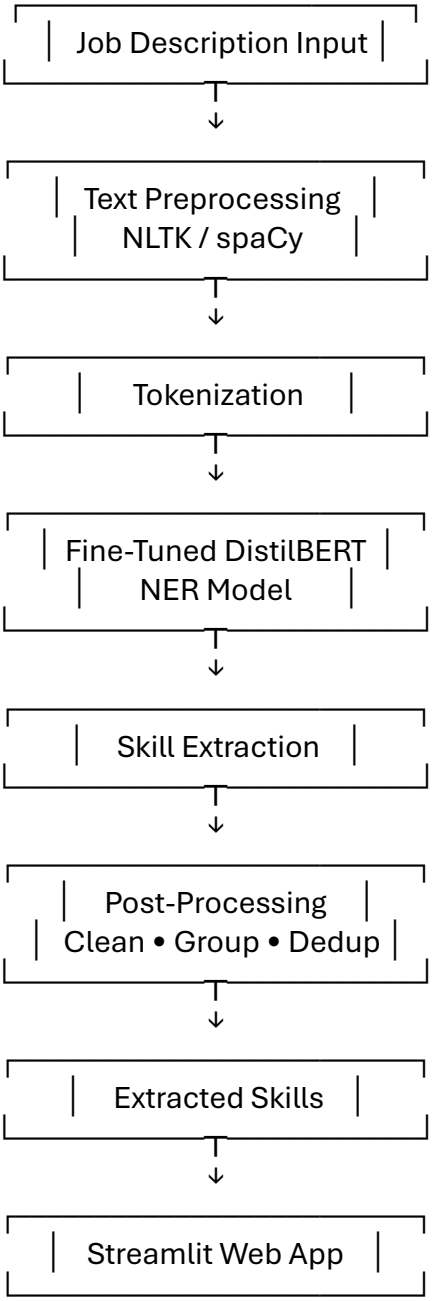
5.2 Project Workflow

1. Job Description Input
The user enters or pastes a job description into the SkillSense web application.
2. Text Preprocessing
The input text is cleaned and prepared for NLP processing using suitable text-processing techniques and libraries.
3. Tokenization
The processed text is converted into tokens using the tokenizer associated with the DistilBERT model.
4. Skill Extraction
The tokenized text is passed to the **fine-tuned DistilBERT-based NER model**, which identifies skill-related entities from the job description.
5. Post-Processing
The predicted entities are cleaned, grouped, and processed to remove unnecessary or duplicate results.
6. Result Generation
The extracted skills are presented in a structured format through the **Streamlit web application**.
7. Model Evaluation
The NER model is evaluated using **Precision, Recall, and F1-Score**. Matplotlib and Seaborn are used to visualize evaluation results.
8. Analytics
The extracted skill information and analytical results can be presented using **Power BI dashboards** for better interpretation.

5.3 Training Methodology

The model development workflow is:
Dataset → Data Preparation → Entity Annotation → Tokenization → DistilBERT Fine-Tuning → Model Evaluation → Saved Model
The trained model is then integrated into the SkillSense application for extracting skills from new job descriptions.

5.4 Overall Architecture Flow



This methodology enables SkillSense to convert unstructured job descriptions into structured and meaningful skill information.

6. DATASET COLLECTION & UNDERSTANDING

6.1 Dataset Source

The dataset used for the SkillSense project contains job-description text and skill-related information required for developing the skill extraction model. The dataset is used for data analysis, preprocessing, annotation, model training, and evaluation.

Dataset Source: ChatGPT

6.2 Dataset Fields

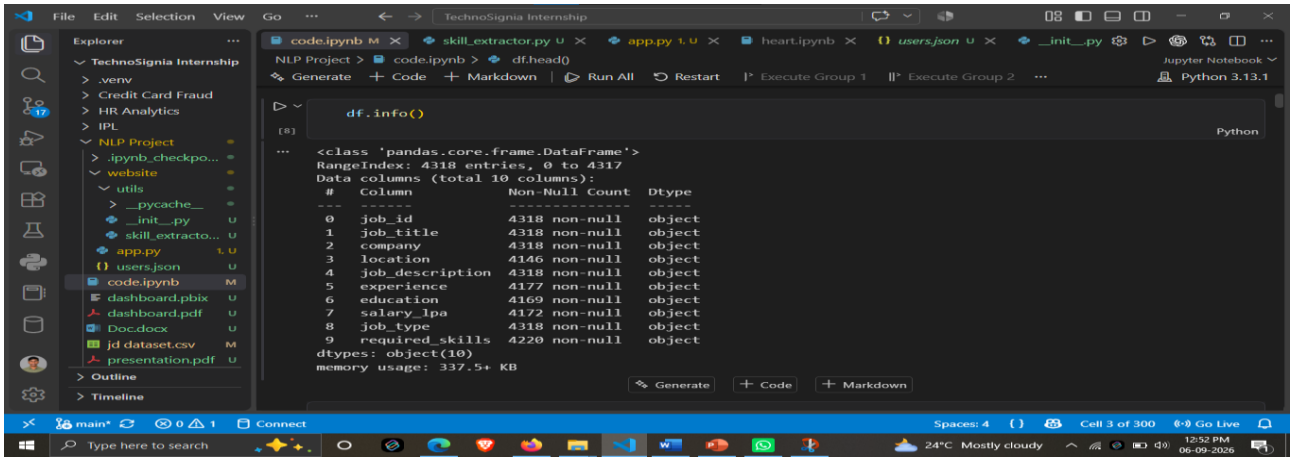
The major fields used in the dataset include:

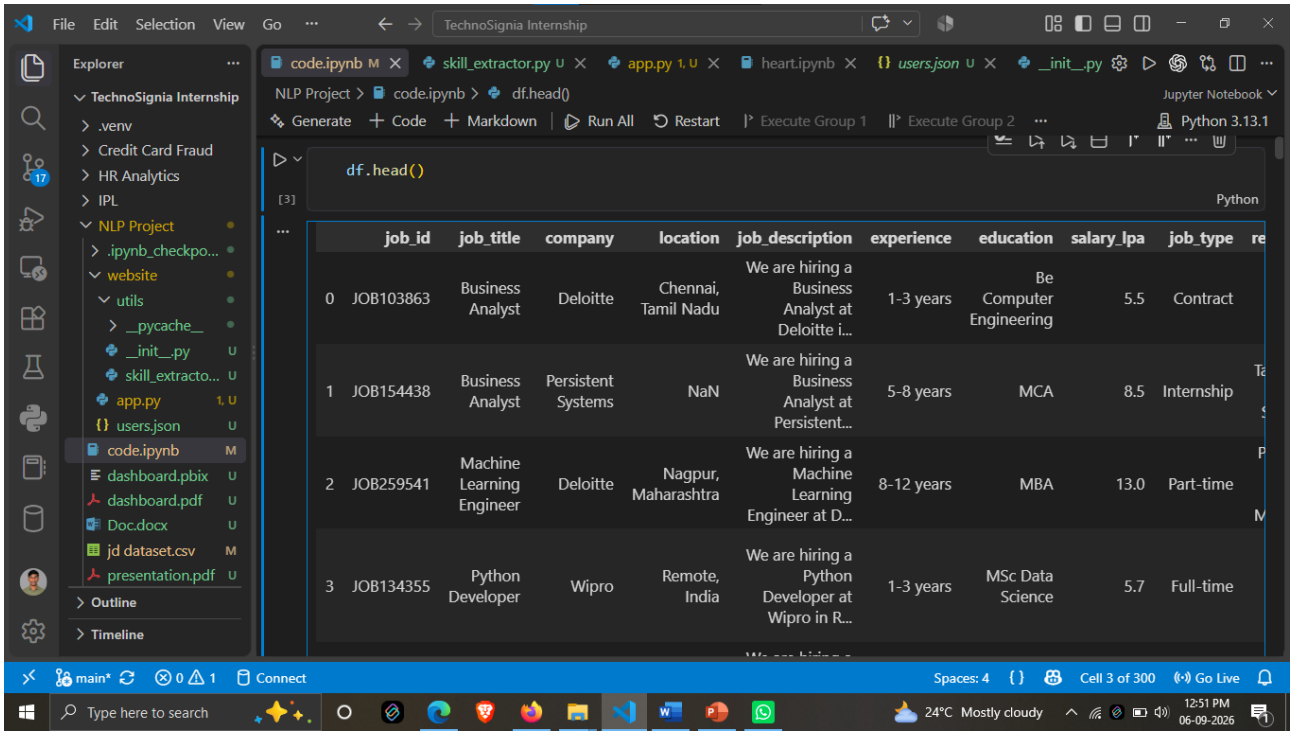
Field	Description
Job Description	Text containing the requirements and responsibilities of a job role
Skills / Annotations	Skill entities identified or annotated within the job description
Job Role	Position or designation associated with the job description

Note: Only the fields actually present in the dataset should be included in the final report.

6.3 Number of Records

The dataset contains approximately **4318 records** used for analysis and model development. The data is divided into suitable training and testing portions to develop and evaluate the skill extraction model.

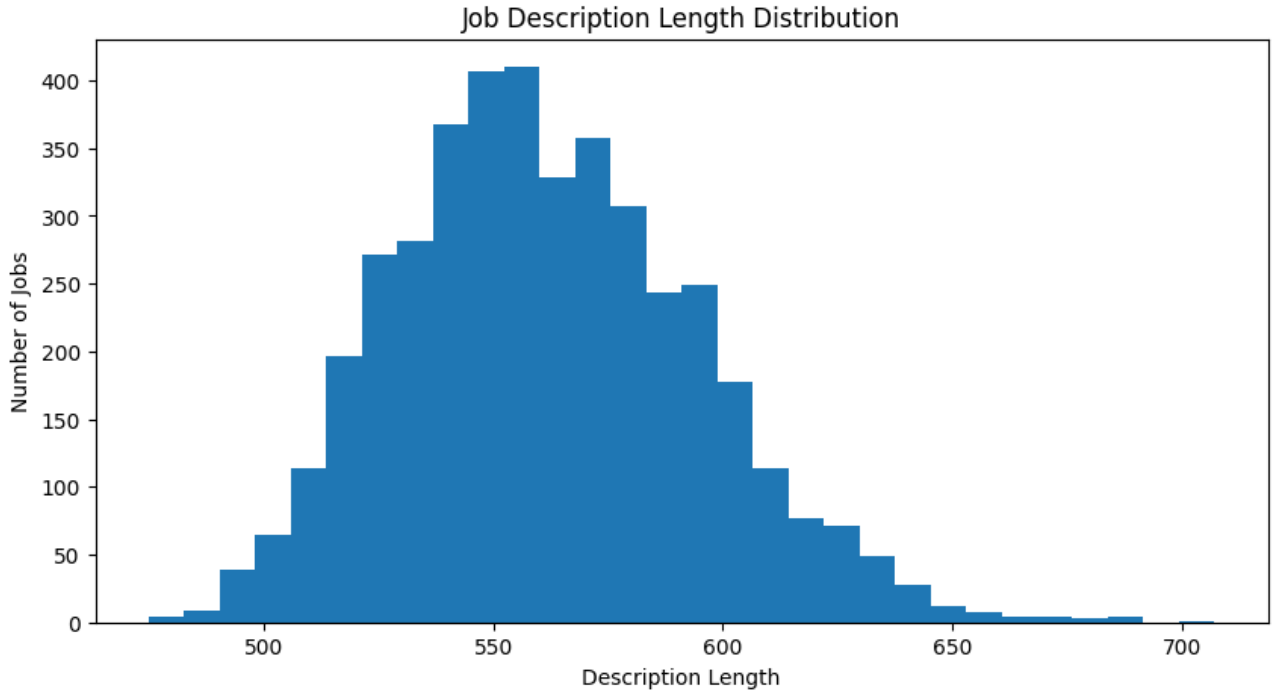




6.4 Data Inspection

Before model development, the dataset was inspected to understand its structure, quality, and contents. The inspection included:

- Checking the number of records and columns.
- Identifying missing or empty values.
- Examining job-description text.
- Checking duplicate records.
- Understanding the distribution of available data.
- Verifying the skill annotations used for NER training.



Basic data analysis and visualization were performed using **Pandas, NumPy, Matplotlib, and Seaborn**.

6.5 Dataset Preparation

After inspection, the relevant data was cleaned and prepared for the NLP pipeline. The job-description text and corresponding skill annotations were converted into a suitable format for training the **DistilBERT-based NER model**.

The prepared dataset was then used for model training, validation, and performance evaluation.

7. EXPLORATORY DATA ANALYSIS (EDA)

7.1 Overview

Exploratory Data Analysis was performed to understand the structure, distribution, and characteristics of the job-description dataset before applying NLP techniques. The analysis included job titles, companies, locations, education requirements, description length, and other important attributes.

7.2 Dataset Overview

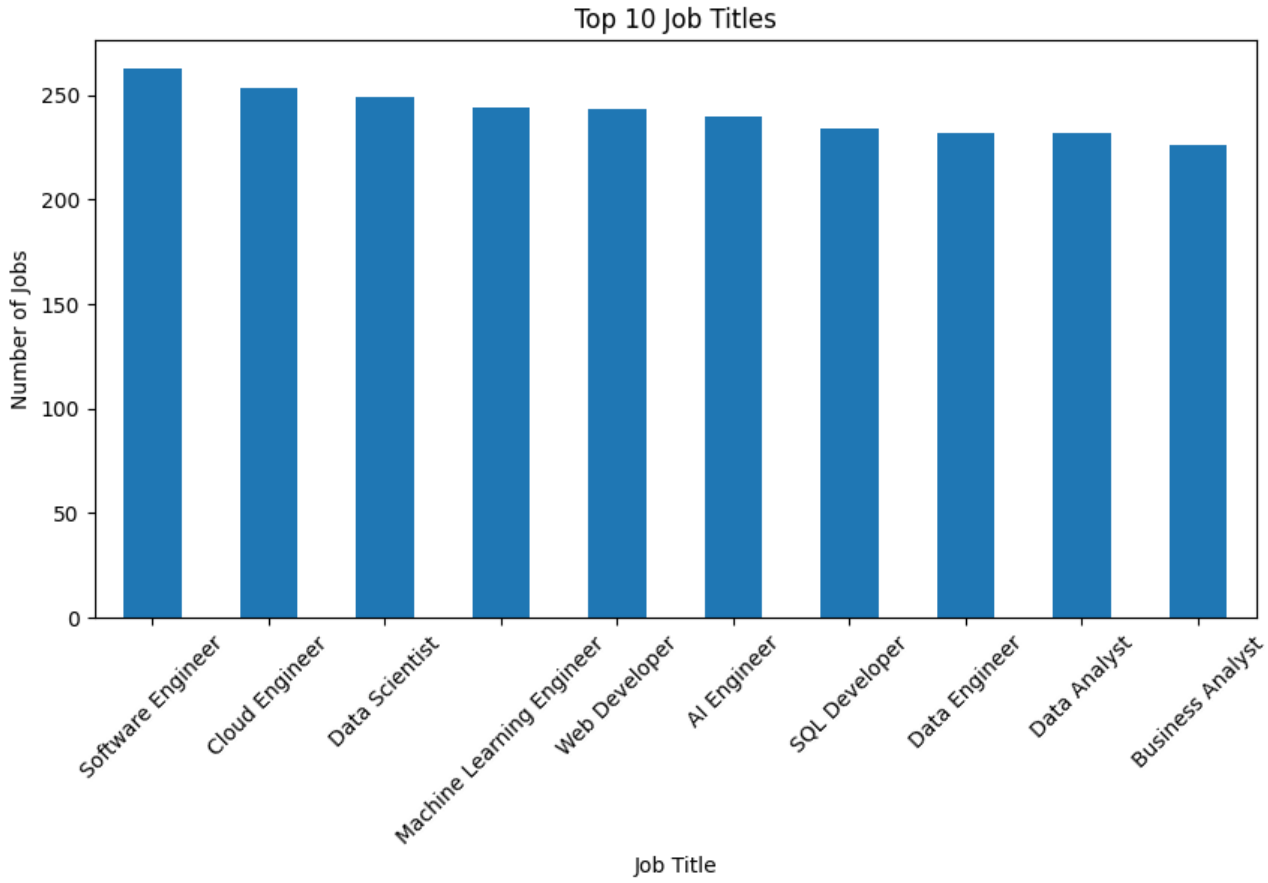
The dataset contains **4,200 job records** and **123 unique job titles**. The analysis also identified **165 unique companies** and **88 unique job locations**.

7.3 Job Title Analysis

The dataset contains a variety of job roles. The most frequently occurring job titles include **Software Engineer, Cloud Engineer, Data Scientist, Machine Learning Engineer, Web Developer, AI Engineer, SQL Developer, Data Engineer, Data Analyst, and Business Analyst**.

The analysis shows that software, data, cloud, and AI-related roles form a significant portion of the dataset.

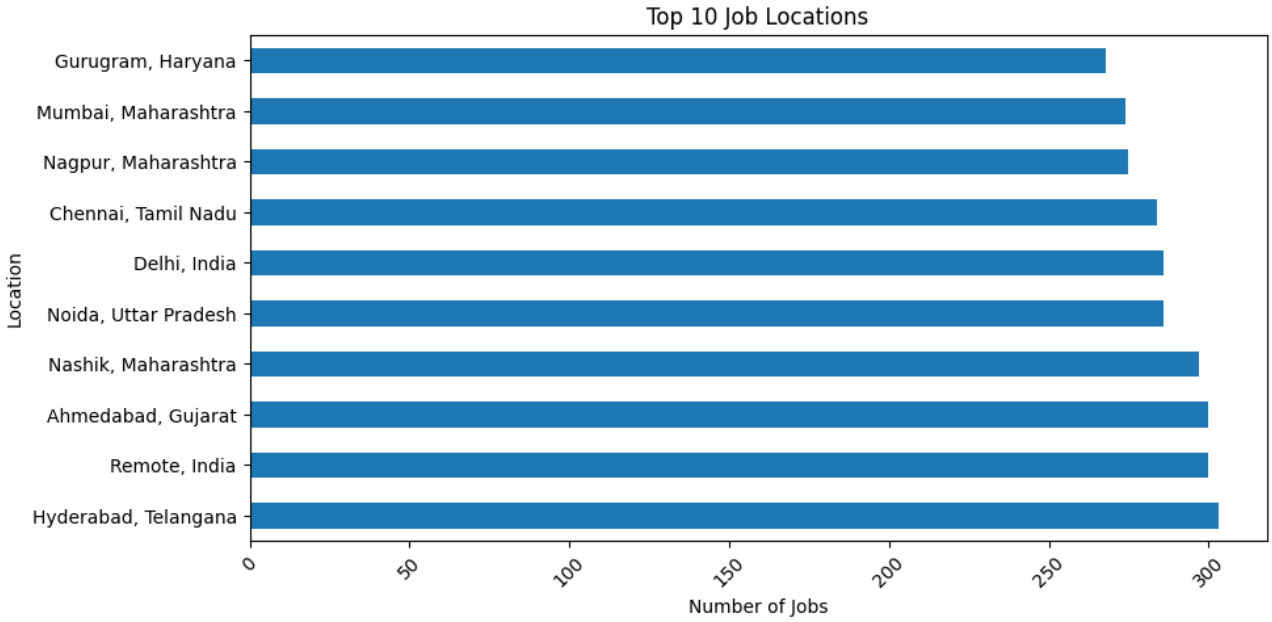
Figure 7.1: Top 10 Job Titles



7.4 Location Analysis

The dataset contains **88 unique locations**. The analysis shows job opportunities distributed across different Indian locations, including **Hyderabad, Remote, Ahmedabad, Nashik, Noida, Delhi, Chennai, Nagpur, Mumbai, and Gurugram**.

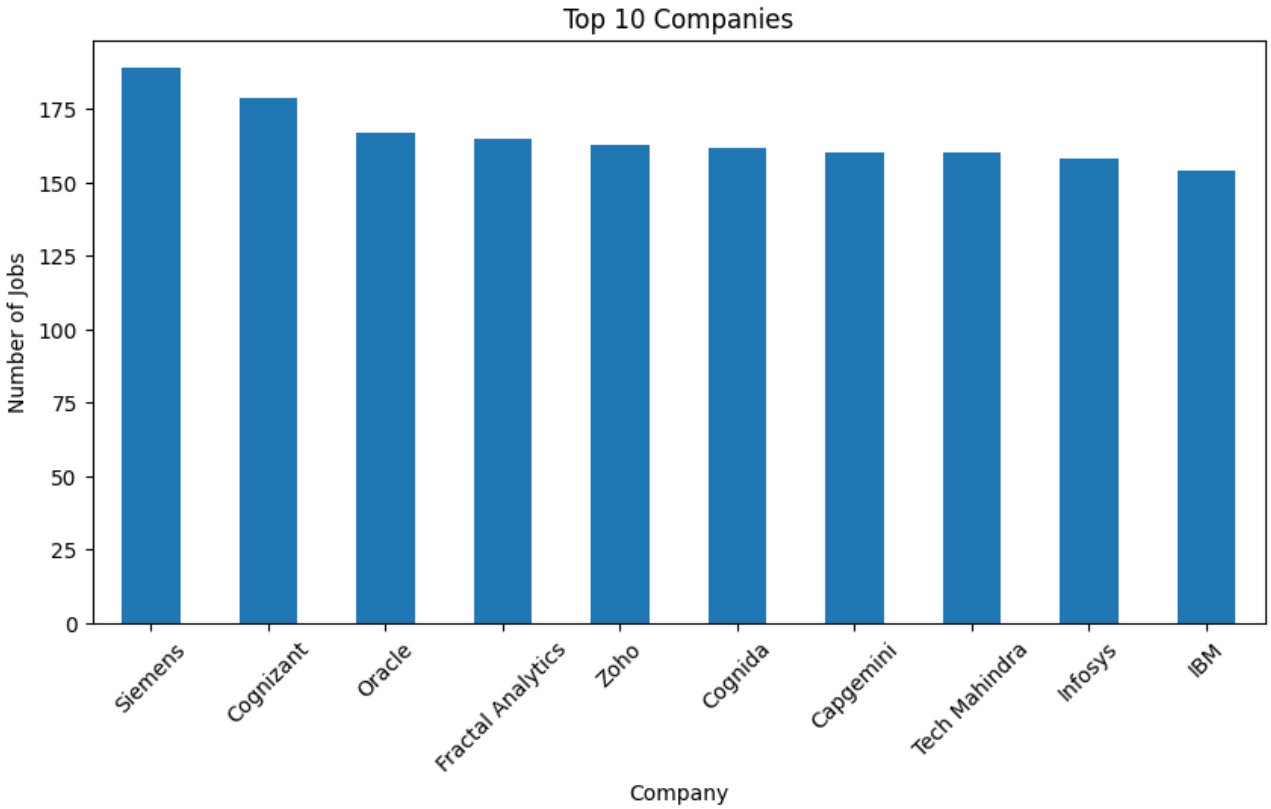
Figure 7.2: Top 10 Job Locations



7.5 Company Analysis

A total of **165 unique companies** are present in the dataset. Companies such as **Siemens, Cognizant, Oracle, Fractal Analytics, Zoho, Cognida, Capgemini, Tech Mahindra, Infosys, and IBM** appear among the frequently represented companies.

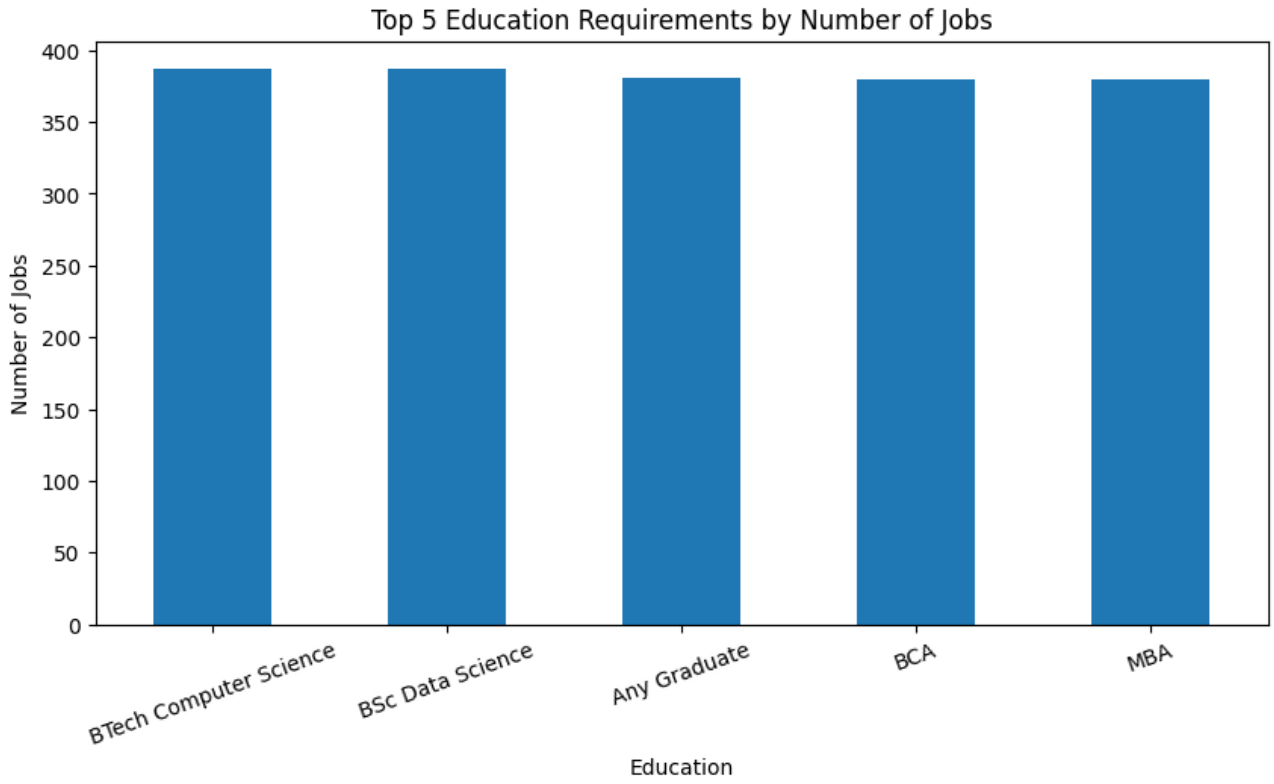
Figure 7.3: Top 10 Companies



7.6 Education Requirement Analysis

The dataset contains different educational requirements for job roles. The frequently observed requirements include **B.Tech Computer Science, B.Sc Data Science, Any Graduate, BCA, and MBA**.

Figure 7.4: Top 5 Education Requirements

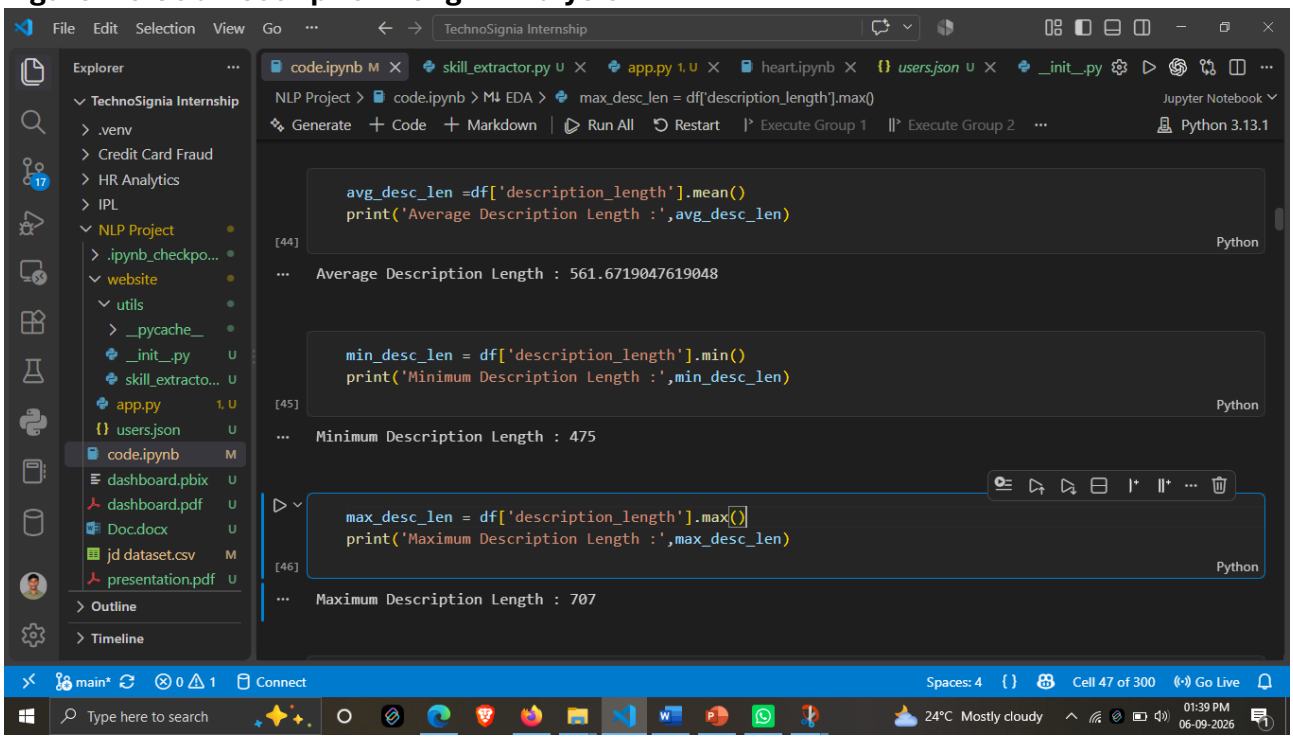


7.7 Job Description Length Analysis

The average job-description length is approximately **561.67**, while the minimum and maximum description lengths are **475** and **707**, respectively.

This analysis helps understand the textual size and consistency of job descriptions before applying NLP processing.

Figure 7.5: Job Description Length Analysis



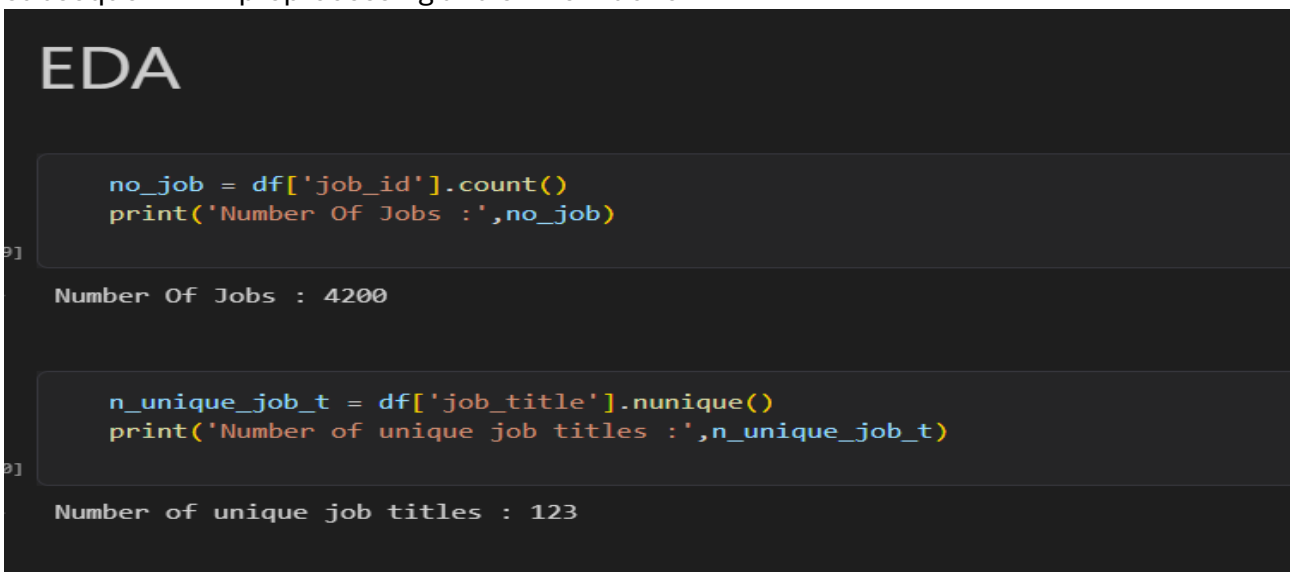
7.8 Key EDA Findings

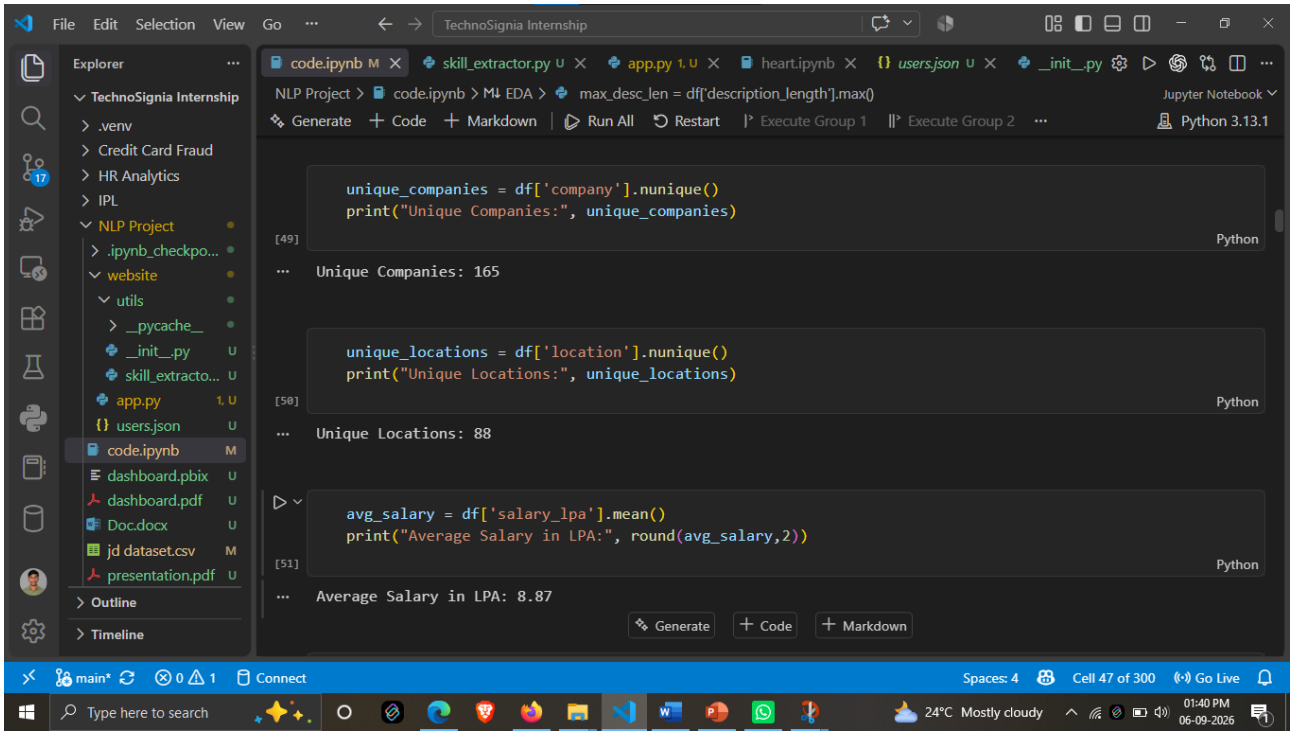
The major findings from the exploratory analysis are:

- Total job records: **4,200**
- Unique job titles: **123**
- Unique companies: **165**

- Unique locations: **88**
- Average description length: **561.67**
- Minimum description length: **475**
- Maximum description length: **707**
- Average salary: **8.87 LPA**

The EDA provided an understanding of the dataset and helped prepare the data for subsequent NLP preprocessing and skill extraction.





8. DATA CLEANING & NLP PREPROCESSING

8.1 Data Cleaning

Data cleaning was performed to improve the quality and consistency of the job-description dataset before applying NLP techniques. Unnecessary or inconsistent text elements were handled during preprocessing.

The main cleaning activities included:

- Handling missing or empty values.
- Removing unnecessary characters and unwanted text.
- Removing duplicate records where applicable.
- Standardizing textual data.
- Preparing job descriptions for further NLP processing.

```
[61]  df['clean_description'] = df['job_description'].apply(clean_text)

[62]  df['clean_description'].head()

...   0    we are hiring a business analyst at deloitte i...
      1    we are hiring a business analyst at persistent...
      2    we are hiring a machine learning engineer at d...
      3    we are hiring a python developer at wipro in r...
      4    we are hiring a software engineer at capgemini...
      Name: clean_description, dtype: object
```

8.2 Tokenization

Tokenization is the process of dividing text into smaller units called **tokens**. These tokens can represent individual words or sub-word units.

For example:

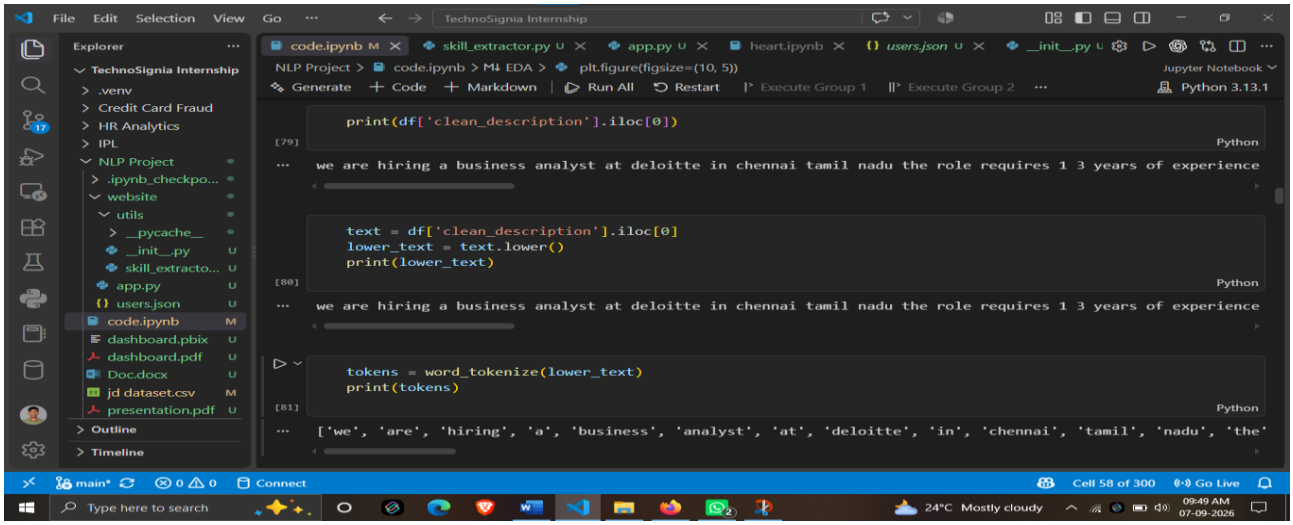
Input:

Experience in Python and Machine Learning

Tokens:

Experience | in | Python | and | Machine | Learning

For the transformer-based model, the final tokenization is performed using the **DistilBERT tokenizer**.



8.3 Stopword Removal

Stopwords are commonly occurring words such as *the, is, in, and, of,* etc., which may provide limited information for some NLP tasks.

Stopword handling can reduce unnecessary textual information during traditional NLP preprocessing. However, for the **DistilBERT-based NER model**, excessive removal of words is avoided because contextual information can be important for identifying entities.

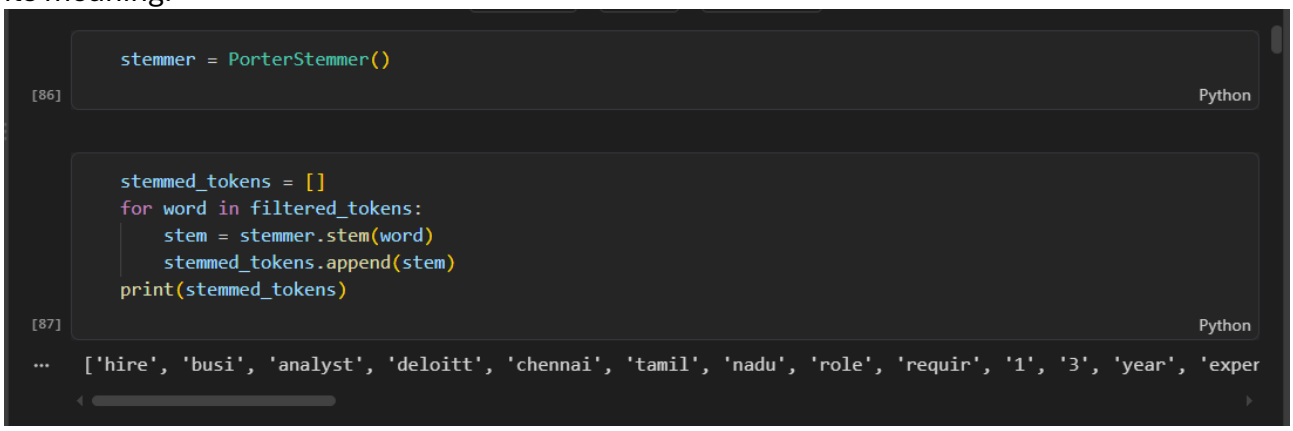
8.4 Stemming

Stemming reduces words to their basic stem by removing prefixes or suffixes.

For example:

learning → learn

Stemming is useful in traditional NLP workflows for reducing variations of words. However, it should be applied carefully in skill extraction because changing a technical term may affect its meaning.



8.5 Lemmatization

Lemmatization converts words into their meaningful base or dictionary form.

For example:

running → run

Lemmatization can help normalize different forms of words while preserving their linguistic meaning. It is mainly useful during text analysis and preprocessing.

8.6 Sentence Segmentation

Sentence segmentation divides a job description into individual sentences. This helps organize the text and supports sentence-level NLP analysis.

For example:

Job Description:

We need a Python developer. Experience with SQL is required.

Segments:

1. We need a Python developer.
2. Experience with SQL is required.

8.7 Preprocessing Workflow

The overall preprocessing workflow can be summarized as:

Raw Job Description → Data Cleaning → Text Normalization → Tokenization → NLP Processing → DistilBERT NER Model

The preprocessing stage prepares the job-description text for effective skill extraction while preserving the contextual information required by the transformer-based NER model.

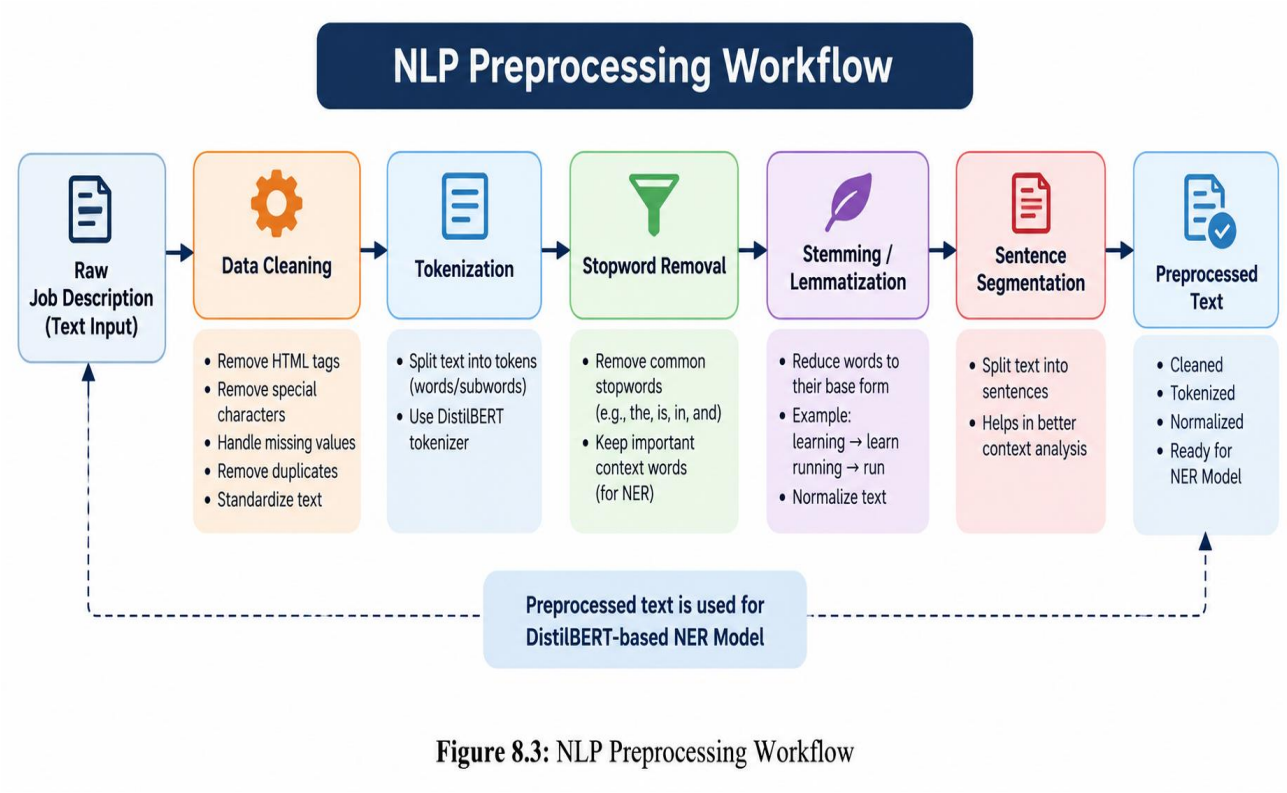


Figure 8.3: NLP Preprocessing Workflow

9. SKILL TAXONOMY DEVELOPMENT

9.1 Skill Dictionary

A **skill dictionary** is a structured collection of skills used to identify relevant technical and professional skills from job descriptions. It provides a reference list against which extracted skill entities can be analyzed and standardized.

The SkillSense skill dictionary may contain programming languages, frameworks, databases, cloud technologies, data science tools, development tools, and other job-related competencies.

Examples include:

- Python
- Java
- SQL
- Machine Learning
- TensorFlow
- React
- AWS
- Docker

9.2 Skill Categories

To organize the extracted skills effectively, skills can be grouped into different categories.

Category	Examples
Programming Languages	Python, Java, C++, JavaScript
Data & Database	SQL, MySQL, PostgreSQL, MongoDB
Machine Learning & AI	Machine Learning, Deep Learning, NLP
Frameworks & Libraries	React, Django, TensorFlow, PyTorch
Cloud & DevOps	AWS, Azure, Docker, Kubernetes
Data Visualization	Power BI, Tableau, Matplotlib
Tools & Technologies	Git, GitHub, Jupyter, VS Code

9.3 Aliases and Synonyms

The same skill can appear in different forms in job descriptions. **Aliases and synonyms** help map these different expressions to the same underlying skill.

Examples:

Mentioned Term	Canonical Skill
Python3, Python 3.x	Python
PostgreSQL, Postgres	PostgreSQL
ML	Machine Learning
NLP, Natural Language Processing	Natural Language Processing
PowerBI, Power BI	Power BI

This normalization helps avoid treating different representations of the same skill as separate skills.

```
rows = []
for category, skills in skill_taxonomy.items():
    for skill in skills:
        aliases = skill_aliases.get(skill, [])
        rows.append({
            "skill": skill,
            "category": category,
            "aliases": ", ".join(aliases)})
skill_df = pd.DataFrame(rows)
```

[74]

```
skill_df.head()
```

[75]

	skill	category	aliases
0	Python	Programming	python3, python 3
1	Java	Programming	
2	C	Programming	
3	C++	Programming	cpp, c plus plus
4	C#	Programming	c sharp, csharp

9.4 Canonical Skills

A **canonical skill** is the standardized name assigned to a skill after processing its different aliases or variations.

For example:

Python3, Python 3.x, Python → Python

ML, Machine Learning → Machine Learning

Canonicalization makes the final extracted skill list more consistent and useful for further analysis.

9.5 Role in SkillSense

The skill taxonomy supports the SkillSense system by:

- Organizing extracted skills into meaningful categories.
- Handling different names and variations of the same skill.
- Reducing duplicate skill representations.
- Standardizing extracted results.
- Improving the consistency of skill-based analysis.

The overall taxonomy process can be represented as:

Raw Skill Mention → Alias/Synonym Detection → Canonical Mapping → Skill Category → Standardized Skill

10. RULE-BASED SKILL EXTRACTION

Rule-Based Skill Extraction is used as an initial approach for identifying skills from job descriptions. It relies on predefined skill dictionaries, regular expressions, and phrase-matching techniques to detect known skill terms in the input text.

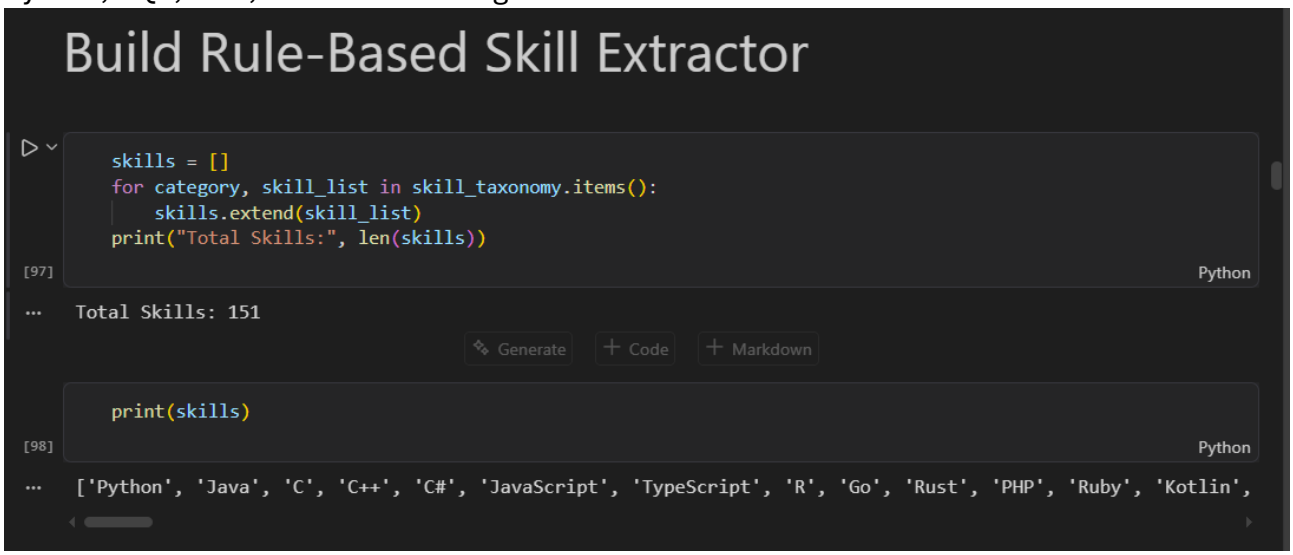
10.1 Dictionary Matching

A predefined **skill dictionary** is used to match known skills with terms present in the job description. When a skill from the dictionary is found in the input text, it is added to the extracted skill list.

This approach is simple, fast, and effective for identifying commonly known skills.

Example:

Python, SQL, Java, Machine Learning



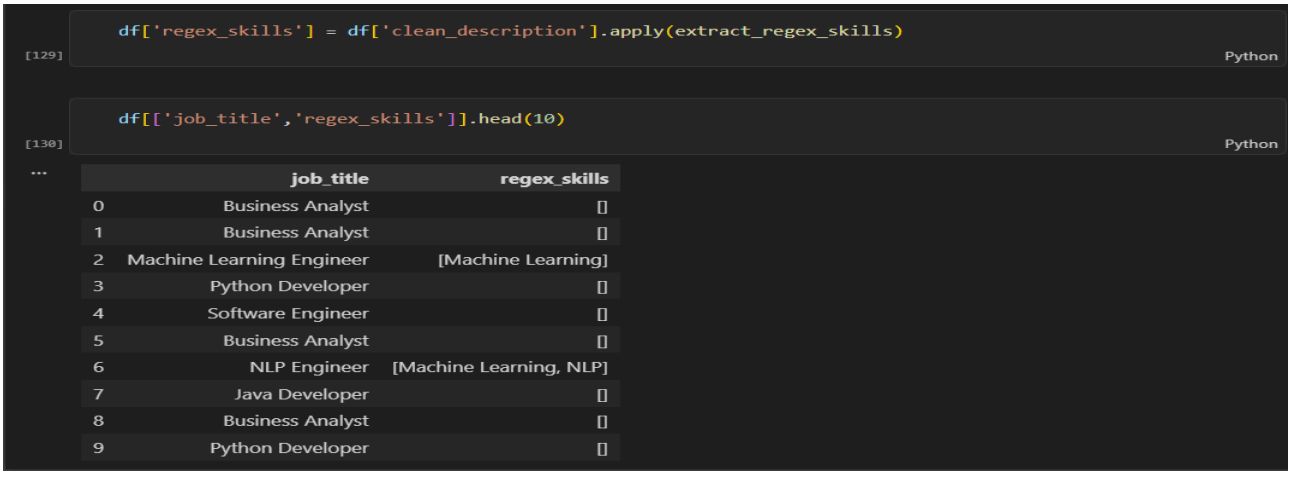
10.2 Regex-Based Matching

Regular Expressions (Regex) are used to identify specific patterns or variations of skill names in the job description. Regex matching is useful when a skill can appear in different formats.

For example, patterns can be designed to identify variations such as:

- Python 3
- Python3
- C++
- C#

Regex-based matching improves the ability to detect skills with different textual representations.



	job_title	regex_skills
0	Business Analyst	[]
1	Business Analyst	[]
2	Machine Learning Engineer	[Machine Learning]
3	Python Developer	[]
4	Software Engineer	[]
5	Business Analyst	[]
6	NLP Engineer	[Machine Learning, NLP]
7	Java Developer	[]
8	Business Analyst	[]
9	Python Developer	[]

10.3 Phrase Matching

Phrase matching is used to identify **multi-word skills or technical terms** that consist of more than one word.

Examples include:

- Machine Learning
- Deep Learning
- Natural Language Processing
- Data Science
- Computer Vision

The complete phrase is matched instead of treating each word separately, which helps maintain the correct meaning of the skill.

```
[138]  df['phrase_skills'] = df['clean_description'].apply(extract_phrase_skills)

[139]  df[['job_title','phrase_skills']].head(10)
```

	job_title	phrase_skills
0	Business Analyst	[]
1	Business Analyst	[]
2	Machine Learning Engineer	[machine learning]
3	Python Developer	[]
4	Software Engineer	[]
5	Business Analyst	[]
6	NLP Engineer	[machine learning]
7	Java Developer	[]
8	Business Analyst	[]
9	Python Developer	[]

10.4 Rule-Based Extraction Workflow

The rule-based extraction process can be summarized as:

Job Description → Dictionary Matching → Regex Matching → Phrase Matching → Extracted Skills

Rule-based extraction provides a baseline approach for skill identification and can also complement the transformer-based **DistilBERT NER model** used in SkillSense.

11. STATISTICAL & SEMANTIC NLP TECHNIQUES

Statistical and semantic NLP techniques are useful for representing and analyzing textual information. In the SkillSense project, these techniques provide different approaches for understanding job-description text and identifying relationships between skills and job requirements.

11.1 TF-IDF

Term Frequency–Inverse Document Frequency (TF-IDF) is a statistical technique used to measure the importance of words within a document relative to a collection of documents. TF-IDF gives higher importance to terms that occur frequently in a particular document but are less common across the complete dataset. It can be useful for identifying important terms in job descriptions.

```
[153]  print("TF-IDF Matrix Shape:", X.shape)                                  Python

       TF-IDF Matrix Shape: (4200, 187)

[154]  terms = vectorizer.get_feature_names_out()
       scores = X.mean(axis=0).A1                                              Python

[155]  tfidf_df = pd.DataFrame({"Term": terms,"TF_IDF_Score": scores})
       tfidf_df = tfidf_df.sort_values("TF_IDF_Score",ascending=False)
       tfidf_df.head(20)                                                       Python
```

	Term	TF_IDF_Score
62	experience	0.097001
159	sql	0.095393
134	python	0.082119
42	data	0.069011

11.2 N-Grams

N-Grams are sequences of consecutive words or tokens extracted from text.

Examples:

- **Unigram:** Machine
- **Bigram:** Machine Learning
- **Trigram:** Natural Language Processing

N-Grams help capture meaningful multi-word expressions that may represent skills or technical concepts.

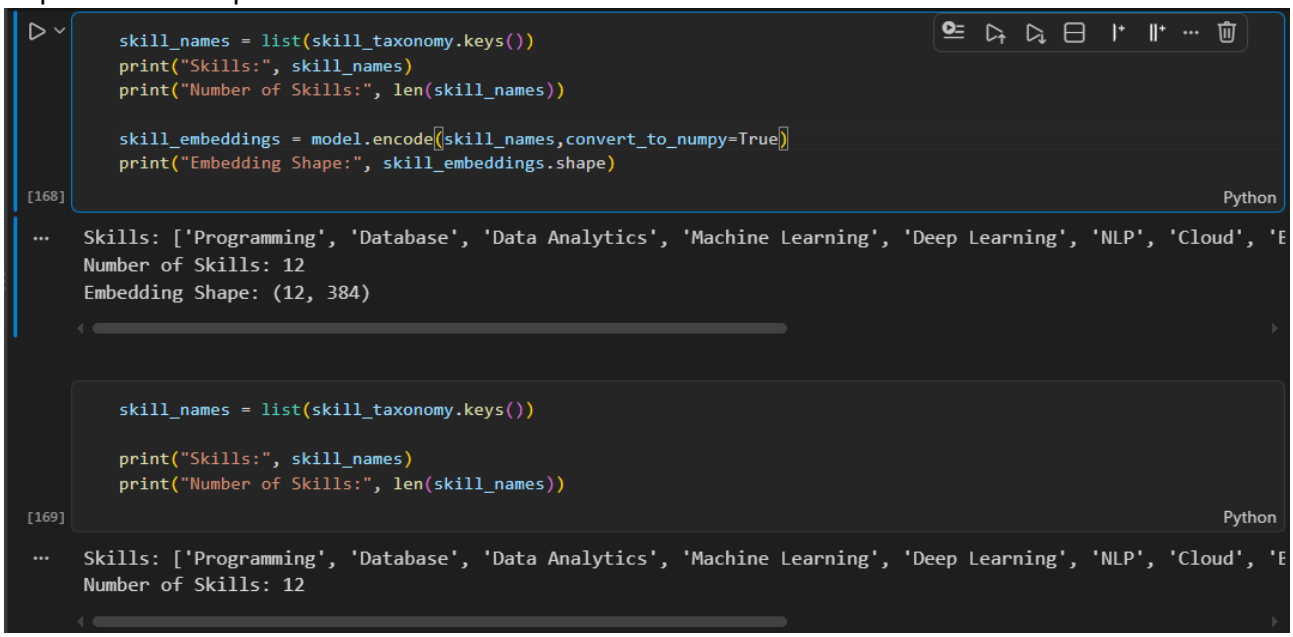
```
[161]  terms = vectorizer.get_feature_names_out()
       scores = X.mean(axis=0).A1
       ngram_df = pd.DataFrame({"N-Gram": terms,"Score": scores})
       ngram_df = ngram_df.sort_values("Score",ascending=False)
       ngram_df.head(20)
```

	N-Gram	Score
2928	experience	0.037681
7490	sql	0.036859
6313	python	0.030905
1587	data	0.026608
8164	technical	0.025753
6862	related	0.025357
2215	docker	0.025117
5099	learning	0.024682
3600	git	0.022906
6213	power	0.022786
2452	engineer	0.021584

11.3 Word Embeddings

Word Embeddings represent words or tokens as numerical vectors in a continuous vector space. Words with similar meanings or contextual usage can have similar vector representations.

Embeddings are useful for capturing semantic relationships between terms and are an important concept in modern NLP and transformer-based models.



11.4 Semantic Similarity

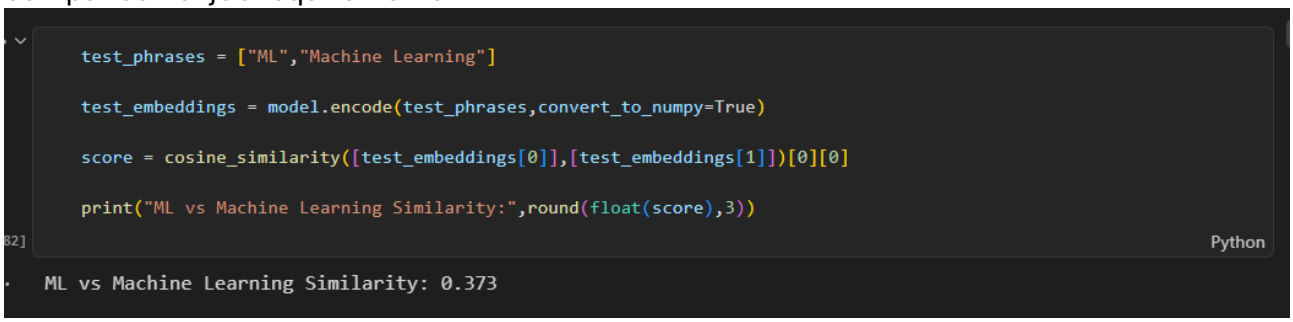
Semantic Similarity measures how closely two words, phrases, or documents are related in meaning.

For example, terms such as:

Machine Learning ↔ ML

may be considered semantically related even though their textual forms are different.

Semantic similarity can therefore support skill matching, synonym identification, and comparison of job requirements.



11.5 Role in SkillSense

These techniques provide different levels of text understanding:

TF-IDF → Term Importance

N-Grams → Phrase Information

Word Embeddings → Semantic Representation

Semantic Similarity → Meaning-Based Comparison

Together, statistical and semantic techniques provide a foundation for analyzing job-description text and understanding relationships between skills.

12. NAMED ENTITY RECOGNITION (NER)

12.1 Standard NER

Named Entity Recognition (NER) is an NLP technique used to identify and classify important entities present in text. Standard NER models commonly identify entities such as **Person, Organization, Location, Date, and other general entities**.

For example:

Input:

Google is hiring a Python Developer in Mumbai.

Possible entities:

- Google → Organization
- Python → Skill/Technology
- Mumbai → Location

Standard NER is useful for general-purpose entity identification but may not specifically recognize all technical skills required from job descriptions.

```
df[["job_title", "standard_entities"]].head(10)
```

	job_title	standard_entities
0	Business Analyst	[(deloitte, ORG), (1 3 years, DATE), (sql jira...
1	Business Analyst	[(5 8 years, DATE), (mca, ORG)]
2	Machine Learning Engineer	[(deloitte, ORG), (8 12 years, DATE)]
3	Python Developer	[(1 3 years, DATE), (msc data, ORG)]
4	Software Engineer	[(bengaluru, GPE), (2 5 years, DATE), (java do...
5	Business Analyst	[(bengaluru, GPE), (8 12 years, DATE), (sql po...
6	NLP Engineer	[(ibm, ORG), (bengaluru, GPE), (3 6 years, DAT...
7	Java Developer	[(siemens, ORG), (1 3 years, DATE), (bsc, ORG)...
8	Business Analyst	[(india, GPE), (5 8 years, DATE), (sql tableau...
9	Python Developer	[(siemens, ORG), (8 12 years, DATE)]

12.2 Custom Entities

For the SkillSense project, the focus is on identifying **job-related skill entities**. Therefore, a custom entity category is used to identify skills such as:

- Python
- SQL
- Machine Learning
- TensorFlow
- Power BI
- Java
- AWS

The custom entity approach allows the model to focus specifically on **skill-related information** rather than only general-purpose entities.

```
df[["job_title","custom_entities"]].head(10)
```

Python

	job_title	custom_entities
0	Business Analyst	[(statistics, SKILL), (sql, DATABASE), (power ...
1	Business Analyst	[(sql, DATABASE), (power bi, BI_TOOL), (tablea...
2	Machine Learning Engineer	[(machine learning, SKILL), (python, PROGRAMMI...
3	Python Developer	[(python, PROGRAMMING_LANGUAGE), (data science...
4	Software Engineer	[(java, PROGRAMMING_LANGUAGE), (sql, DATABASE)...
5	Business Analyst	[(statistics, SKILL), (sql, DATABASE), (power ...
6	NLP Engineer	[(machine learning, SKILL), (nlp, SKILL), (pyt...
7	Java Developer	[(java, PROGRAMMING_LANGUAGE), (spring, WEB_TE...
8	Business Analyst	[(statistics, SKILL), (sql, DATABASE), (power ...
9	Python Developer	[(python, PROGRAMMING_LANGUAGE), (sql, DATABAS...

12.3 Custom NER Approach

SkillSense uses a **custom NER approach based on a fine-tuned DistilBERT model**. The model is trained on job-description data containing skill-related annotations.

The workflow is:

Annotated Job Dataset → Tokenization → DistilBERT Fine-Tuning → Custom NER Model → Skill Entity Prediction

During prediction, the model analyzes the context of the job description and identifies text spans representing relevant skills.

For example:

Input:

The candidate should have experience in Python, SQL and Machine Learning.

Extracted Skills:

Python | SQL | Machine Learning

The custom NER approach enables SkillSense to perform domain-specific skill extraction from unstructured job descriptions.

13. MACHINE LEARNING MODEL TRAINING

13.1 Training Data

The prepared job-description dataset was used for training and evaluating the machine learning model. The dataset was divided into suitable **training and testing sets** to evaluate the model on unseen data.

The training data contains job-related textual information and the corresponding target information required for the selected machine learning task.

```
X = training_df["text"]
y = training_df["label"]
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=42,stratify=y)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Python

```
Training samples: 30
Testing samples: 14
```

13.2 Feature Engineering

Since machine learning models cannot directly process raw text, the textual job descriptions were converted into numerical features.

TF-IDF (Term Frequency–Inverse Document Frequency) was used to transform the job-description text into numerical feature vectors. In the project, the generated TF-IDF matrix contains **4,200 documents and 187 features**.

This representation allows the machine learning model to identify patterns and relationships within the textual data.

13.3 TF-IDF + Machine Learning Model

The machine learning pipeline combines **TF-IDF vectorization** with a machine learning model.

The workflow is:

Job Description → Text Preprocessing → TF-IDF Vectorization → ML Model → Prediction

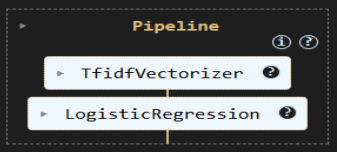
TF-IDF converts the text into numerical vectors, which are then provided as input to the machine learning model for training and prediction.

```
model = Pipeline([("tfidf", TfidfVectorizer(lowercase=True,ngram_range=(1, 2))),
    ("classifier", LogisticRegression(max_iter=1000))])
```

Python

```
model.fit(X_train, y_train)
```

Python




```
y_pred = model.predict(X_test)
print("Accuracy:",accuracy_score(y_test, y_pred))
```

Python

```
Accuracy: 0.21428571428571427
```

13.4 Classification Pipeline

The classification pipeline consists of the following stages:

Job Description

↓

Text Preprocessing

↓

TF-IDF Vectorization

↓

Training / Testing Split

↓

Machine Learning Model

↓

Prediction

↓

Performance Evaluation

The trained model is evaluated using appropriate performance metrics such as **Accuracy, Precision, Recall, and F1-Score**, depending on the classification task.

```
print(classification_report(y_test, y_pred))
```

```
[ ]
```

```
...                          precision    recall  f1-score   support

                BIG_DATA       0.00      0.00      0.00         1
                BI_TOOL        0.00      0.00      0.00         1
          CLOUD_PLATFORM       0.00      0.00      0.00         1
                DATABASE       0.00      0.00      0.00         2
            DATA_LIBRARY       0.00      0.00      0.00         1
             DEVOPS_TOOL       0.00      0.00      0.00         1
            ML_FRAMEWORK       0.00      0.00      0.00         1
      PROGRAMMING_LANGUAGE       0.00      0.00      0.00         1
                   SKILL       0.21      1.00      0.35         3
          WEB_TECHNOLOGY       0.00      0.00      0.00         2

                accuracy                           0.21        14
               macro avg       0.02      0.10      0.04        14
            weighted avg       0.05      0.21      0.08        14
```

13.5 Model Training Process

The overall training process includes:

1. Preparing the cleaned dataset.
2. Splitting the data into training and testing sets.
3. Applying TF-IDF vectorization.
4. Training the selected machine learning model.
5. Generating predictions on test data.
6. Evaluating model performance.
7. Comparing the obtained results.

14. Transformer-Based Model

14.1 Transformer Model Overview

Transformer-based models are modern NLP models designed to understand the contextual relationships between words in a text. Popular transformer models include **BERT, DistilBERT, and RoBERTa**.

For the SkillSense project, **DistilBERT** was selected for skill extraction because it provides contextual language understanding while being lighter and faster than the original BERT model.

```
from transformers import pipeline

transformer_ner = pipeline(
    "token-classification",
    model="dslim/distilbert-NER",
    aggregation_strategy="simple",
    device=-1)
print("DistilBERT NER model loaded successfully.")
```

Python

```
Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"b767b19a5cbd421283f6324c3afcf777","version_major":2,"version_minor":0}

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"bb10049db3b144c0a103bdea9111d761","version_major":2,"version_minor":0}

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"ed432f58b52d49ed8275263fc132e73a","version_major":2,"version_minor":0}

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"7af22259da34447da31f1009dc74b94e","version_major":2,"version_minor":0}

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id":"c69b79d6f79c487e879d186638f39137","version_major":2,"version_minor":0}
```

14.2 DistilBERT Model Approach

SkillSense uses a **fine-tuned DistilBERT model for Named Entity Recognition (NER)**. The pretrained DistilBERT model is adapted to the specific task of identifying skill-related entities from job descriptions.

The model learns from annotated job-description data where relevant skills are marked as target entities.

The approach is:

Pretrained DistilBERT → Fine-Tuning with Skill Dataset → Custom NER Model → Skill Prediction

```
text = df["clean_description"].iloc[0]
entities = transformer_ner(text)
for entity in entities:
    print(
        entity["word"],
        "→",
        entity["entity_group"],
        round(float(entity["score"]), 3)
    )
```

```
del → ORG 0.943
##oit → ORG 0.979
##te → ORG 0.984
```

```
def transformer_entities(text):
    entities = transformer_ner(str(text))
    return [
        (entity["word"],
        entity["entity_group"],
        round(float(entity["score"]), 3))
        for entity in entities]
```

14.3 Skill Extraction Process

When a new job description is provided, the extraction process follows these steps:

1. **Input:** User provides a job description.
2. **Tokenization:** The text is converted into model-compatible tokens.
3. **Contextual Processing:** DistilBERT analyzes the relationship between tokens and their surrounding context.
4. **Entity Prediction:** The NER layer predicts whether tokens belong to a skill entity.
5. **Entity Grouping:** Related tokens are combined to form complete skill phrases.
6. **Post-Processing:** Duplicate or unnecessary results are cleaned.
7. **Output:** The final extracted skills are displayed to the user.

```
df[["job_description", "transformer_entities"]].head(10)
```

	job_description	transformer_entities
0	We are hiring a Business Analyst at Deloitte i...	[(del, ORG, 0.943), (##oit, ORG, 0.979), (##te...
1	We are hiring a Business Analyst at Persistent...	[(##dia, LOC, 0.624)]
2	We are hiring a Machine Learning Engineer at D...	[(del, ORG, 0.908), (##oit, ORG, 0.96), (##te,...
3	We are hiring a Python Developer at Wipro in R...	[(##ip, ORG, 0.915), (##ro, ORG, 0.927), (##di...
4	We are hiring a Software Engineer at Capgemini...	[(cap, ORG, 0.58), (##ge, ORG, 0.914), (##mini...
5	We are hiring a Business Analyst at Persistent...	[(ben, LOC, 0.609), (##gal, LOC, 0.875), (##ur...
6	We are hiring a NLP Engineer at IBM in Bengalu...	[(i, ORG, 0.592), (##m, ORG, 0.745), (ben, LOC...
7	We are hiring a Java Developer at Siemens in R...	[(##dia, LOC, 0.52)]
8	We are hiring a Business Analyst at TCS in Del...	[(del, LOC, 0.774), (##hi, LOC, 0.935)]
9	We are hiring a Python Developer at Siemens in...	[(##ens, LOC, 0.404), (##g, LOC, 0.487), (##pu...

Example

Input:

Experience in Python, SQL and Machine Learning is required.

Extracted Skills:

Python | SQL | Machine Learning

14.4 Overall Extraction Pipeline

Job Description

↓

DistilBERT Tokenizer

↓

Fine-Tuned DistilBERT NER

↓

Entity Prediction

↓

Skill Entity Grouping

↓

Post-Processing

↓

Extracted Skills

The transformer-based approach enables SkillSense to identify skills based on **context**, making it more suitable for job-description analysis than simple keyword matching alone.

15. MODEL EVALUATION & COMPARISON

The developed models were evaluated using standard performance metrics to measure their effectiveness in identifying and classifying relevant information from job-description data. The main evaluation metrics used are **Precision, Recall, F1-Score, and Accuracy**. A **Confusion Matrix** was also used to analyze classification performance.

15.1 Precision

Precision measures how many of the entities predicted by the model are actually correct.

Precision=TPTP+FPPrecision = \frac{TP}{TP + FP}

A higher precision indicates fewer false-positive predictions.

15.2 Recall

Recall measures how many of the actual relevant entities were correctly identified by the model.

Recall=TPTP+FNRecall = \frac{TP}{TP + FN}

A higher recall indicates that the model is able to identify more relevant skills.

15.3 F1-Score

F1-Score is the harmonic mean of Precision and Recall.

F1=2×Precision×RecallPrecision+RecallF1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}

It provides a balanced measure when both precision and recall are important.

15.4 Accuracy

Accuracy represents the proportion of correct predictions among all predictions.

Accuracy=TP+TNTP+TN+FP+FNAccuracy = \frac{TP + TN}{TP + TN + FP + FN}

Accuracy is mainly useful for evaluating classification models where the class distribution is reasonably balanced.

15.5 Confusion Matrix

A **Confusion Matrix** provides a detailed view of classification results by showing correct and incorrect predictions for different classes.

It consists of:

- **True Positive (TP)**
- **True Negative (TN)**
- **False Positive (FP)**
- **False Negative (FN)**

The confusion matrix helps identify which classes are being predicted correctly and where classification errors occur.

FINAL SKILL EXTRACTION COMPARISON

==========

	Method	Accuracy	Precision	Recall	F1 Score
0	Dictionary	0.620	0.583	0.620	0.601
1	Regex	0.052	0.567	0.052	0.096
2	Semantic Similarity	0.036	0.852	0.036	0.069
3	Transformer	0.636	0.916	0.636	0.751

15.6 Model Comparison

Different approaches used in the project were compared based on their evaluation performance. The comparison helps determine which approach provides better results for the given task.

A typical comparison table can be presented as:

```
TF-IDF + Logistic Regression
--------------------------------
Accuracy: 0.21428571428571427

Classification Report:
                      precision    recall  f1-score   support

             BIG_DATA       0.00      0.00      0.00         1
              BI_TOOL       0.00      0.00      0.00         1
       CLOUD_PLATFORM       0.00      0.00      0.00         1
             DATABASE       0.00      0.00      0.00         2
        DATA_LIBRARY        0.00      0.00      0.00         1
         DEVOPS_TOOL        0.00      0.00      0.00         1
        ML_FRAMEWORK        0.00      0.00      0.00         1
PROGRAMMING_LANGUAGE        0.00      0.00      0.00         1
               SKILL        0.21      1.00      0.35         3
       WEB_TECHNOLOGY       0.00      0.00      0.00         2

            accuracy                           0.21        14
           macro avg       0.02      0.10      0.04        14
        weighted avg       0.05      0.21      0.08        14
```

15.7 Evaluation Summary

The evaluation results provide a quantitative comparison of the implemented approaches. Precision, Recall, and F1-Score are particularly important for skill extraction because the system needs to identify relevant skills while minimizing incorrect predictions.

16. ERROR ANALYSIS

Error analysis was performed to understand the limitations and incorrect predictions produced during skill extraction. It helps identify areas where the NLP models and extraction rules can be improved.

16.1 False Positives

A **false positive** occurs when the system identifies a word or phrase as a skill even though it is not a relevant skill.

Example:

A common word or organization name may incorrectly be identified as a skill entity.

16.2 False Negatives

A **false negative** occurs when an actual skill mentioned in the job description is not detected by the system.

Example:

A less common technology or newly introduced skill may be missed if it is not sufficiently represented in the training data or skill dictionary.

```
error_df.head(10)
```

	Text	Actual	Predicted	Error
0	we are hiring a business analyst at deloitte i...	sql	None	False Negative
1	we are hiring a business analyst at deloitte i...	statistics	None	False Negative
2	we are hiring a business analyst at deloitte i...	power bi	None	False Negative
3	we are hiring a business analyst at deloitte i...	jira	None	False Negative
4	we are hiring a business analyst at persistent...	power bi	None	False Negative
5	we are hiring a business analyst at persistent...	statistics	None	False Negative
6	we are hiring a business analyst at persistent...	sql	None	False Negative
7	we are hiring a business analyst at persistent...	tableau	None	False Negative
8	we are hiring a business analyst at persistent...	excel	None	False Negative
9	we are hiring a machine learning engineer at d...	scikit-learn	None	False Negative

16.3 Partial Entities

Sometimes the system identifies only part of a multi-word skill instead of the complete entity.

Example:

Machine Learning → Machine

instead of:

Machine Learning

This can occur because of token-level prediction or incorrect entity grouping.

```
partial_cases = error_df[error_df["Error Category"] == "Partial Entity"]
partial_cases[["Text", "Actual", "Predicted", "Error Category"]].head(20)
```

	Text	Actual	Predicted	Error Category
0	we are hiring a business analyst at deloitte i...	{sql, statistics, power bi, jira}	{Power BI, SQL}	Partial Entity
1	we are hiring a business analyst at persistent...	{power bi, statistics, sql, tableau, excel}	{Excel, Power BI, Tableau, SQL}	Partial Entity
2	we are hiring a machine learning engineer at d...	{scikit-learn, machine learning, aws, tensorfl...	{SQL, AWS, TensorFlow, Scikit-learn, Python}	Partial Entity
3	we are hiring a python developer at wipro in r...	{rest api, fastapi, python, django, docker, git}	{Docker, Python, REST API, Django}	Partial Entity
4	we are hiring a software engineer at capgemini...	{c++ | docker | rest api | sql | agile | java}	{Java, SQL, REST API, Docker}	Partial Entity
5	we are hiring a business analyst at persistent...	{power bi, statistics, sql, tableau, excel, jira}	{Excel, Power BI, Tableau, SQL}	Partial Entity
6	we are hiring a nlp engineer at ibm in bengalu...	{machine learning, bert, python, transformers,...	{Transformers, Python, BERT}	Partial Entity

16.4 Synonyms and Skill Variations

The same skill can appear in different forms in job descriptions.
Examples:

- ML → Machine Learning
- NLP → Natural Language Processing
- Postgres → PostgreSQL

If these variations are not properly mapped, the system may treat them as different skills or fail to recognize them.

16.5 Spelling Issues

Spelling mistakes, abbreviations, and inconsistent formatting can affect skill extraction.
Examples:

- Pythn instead of Python
- PowerBI instead of Power BI
- Tensor flow instead of TensorFlow

Such variations may result in missed or incorrectly extracted skills.

16.6 Error Analysis Summary

The major sources of errors are:

Error Type	Effect
False Positive	Incorrect skill identification
False Negative	Actual skill is missed
Partial Entity	Incomplete skill extraction
Synonyms	Same skill treated differently
Spelling Issues	Skill may not be recognized

Error analysis indicates that **better training data, improved entity grouping, skill normalization, synonym handling, and spelling correction** can improve the overall quality of skill extraction.

17. SKILL NORMALIZATION & CATEGORIZATION

Skill normalization is the process of converting different representations of the same skill into a common standardized form. This helps SkillSense produce consistent and non-duplicated skill results.

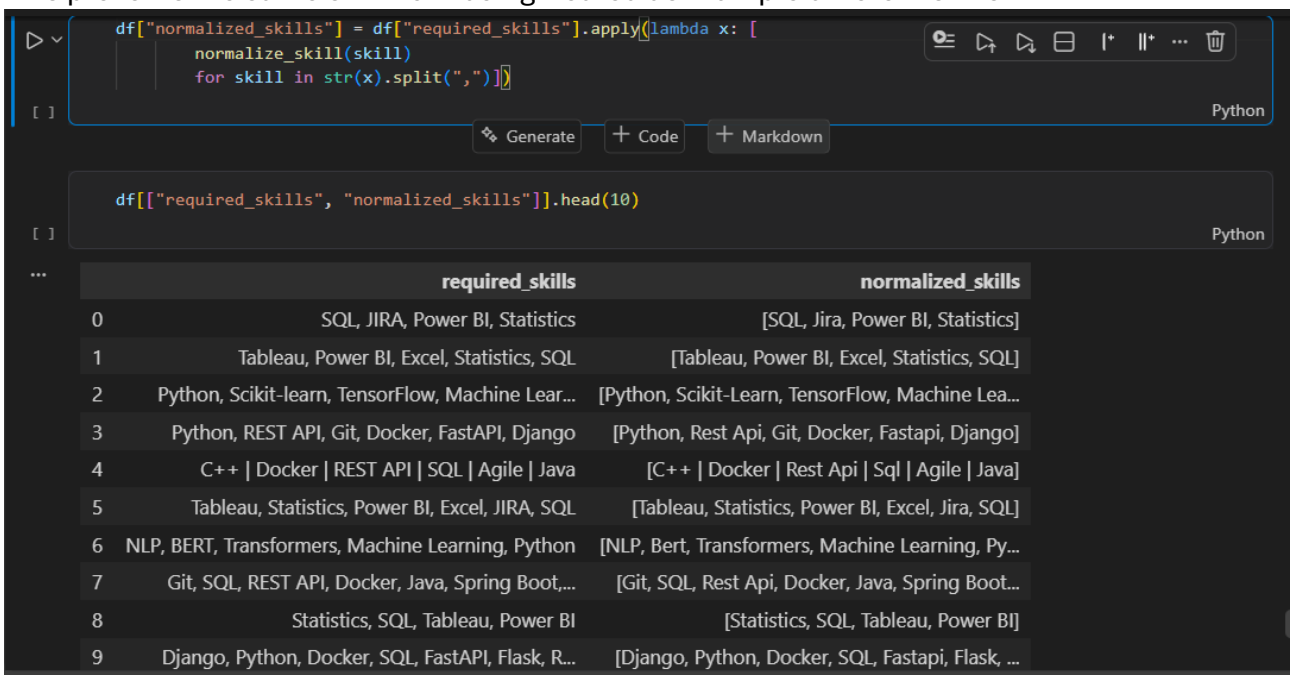
17.1 Alias Matching

The system handles different names, abbreviations, and variations of skills by mapping them to a common skill.

Examples:

Alias / Variation	Canonical Skill
ML	Machine Learning
NLP	Natural Language Processing
Python 3	Python
Postgres	PostgreSQL
PowerBI	Power BI

This prevents the same skill from being treated as multiple different skills.



```
df["normalized_skills"] = df["required_skills"].apply(lambda x: [
        normalize_skill(skill)
        for skill in str(x).split(",")])
```

Python

Generate + Code + Markdown

```
df[["required_skills", "normalized_skills"]].head(10)
```

Python

	required_skills	normalized_skills
0	SQL, JIRA, Power BI, Statistics	[SQL, Jira, Power BI, Statistics]
1	Tableau, Power BI, Excel, Statistics, SQL	[Tableau, Power BI, Excel, Statistics, SQL]
2	Python, Scikit-learn, TensorFlow, Machine Lear...	[Python, Scikit-Learn, TensorFlow, Machine Lea...
3	Python, REST API, Git, Docker, FastAPI, Django	[Python, Rest Api, Git, Docker, Fastapi, Django]
4	C++ | Docker | REST API | SQL | Agile | Java	[C++ | Docker | Rest Api | Sql | Agile | Java]
5	Tableau, Statistics, Power BI, Excel, JIRA, SQL	[Tableau, Statistics, Power BI, Excel, Jira, SQL]
6	NLP, BERT, Transformers, Machine Learning, Python	[NLP, Bert, Transformers, Machine Learning, Py...
7	Git, SQL, REST API, Docker, Java, Spring Boot,...	[Git, SQL, Rest Api, Docker, Java, Spring Boot...
8	Statistics, SQL, Tableau, Power BI	[Statistics, SQL, Tableau, Power BI]
9	Django, Python, Docker, SQL, FastAPI, Flask, R...	[Django, Python, Docker, SQL, Fastapi, Flask, ...

17.2 Canonical Skills

A **canonical skill** is the standardized name assigned to a skill after processing its aliases and variations.

For example:

ML → Machine Learning

Python3 → Python

PowerBI → Power BI

Canonicalization makes the extracted output cleaner and more consistent.

17.3 Category Mapping

After normalization, skills can be mapped to appropriate categories based on their technical domain.

Skill	Category
Python	Programming Language
SQL	Database
TensorFlow	Machine Learning / AI

Skill	Category
React	Framework
AWS	Cloud
Power BI	Data Visualization

This categorization makes the extracted skills easier to analyze and visualize.

```python
def normalize_skill(skill):
    skill = str(skill).strip().lower()
    if skill in skill_normalization:
        return skill_normalization[skill]
    return skill.title()
```

```python
test_skills = [
    "Python",
    "Python3",
    "Python 3",
    "Python programming",
    "PowerBI",
    "Power BI",
    "Power-BI"]
for skill in test_skills:
    print(skill, "→", normalize_skill(skill))
```

```
Python → Python
Python3 → Python
Python 3 → Python
Python programming → Python
PowerBI → Power BI
```

17.4 Normalization Workflow

The complete process can be represented as:

Extracted Skill

↓

Alias / Variation Matching

↓

Canonical Skill

↓

Category Mapping

↓

Normalized Skill Output

Skill normalization and categorization improve the consistency of extracted results and make them more useful for **skill analysis, comparison, and recruitment-related applications**.

18. DASHBOARD & DATA VISUALIZATION

18.1 Dashboard Overview

A Power BI dashboard was developed to present the job-market and skill-related analysis in an interactive and easy-to-understand format. The dashboard combines key performance indicators, skill demand, job-role analysis, skill categories, and salary information.

18.2 Key Performance Indicators (KPIs)

The dashboard provides important summary indicators:

- **Total Jobs:** 4,292
- **Unique Skills:** 58
- **Top Skill:** SQL
- **Top Job Role:** Software Engineer
- **Most Required Technology:** SQL
- **Top Skill Category:** Machine Learning

18.3 Top Skills

The **Top 20 Most Demanded Skills** visualization shows the most frequently required skills in the dataset. **SQL** and **Python** are among the most demanded skills, followed by technologies such as Docker, Git, Machine Learning, AWS, REST API, TensorFlow, and Power BI.

18.4 Skill Demand by Job Role

The dashboard provides a comparison of skill demand across different job roles, including **Software Engineer, Cloud Engineer, AI Engineer, Data Scientist, and Web Developer**. This helps understand which technologies are associated with different job positions.

18.5 Skill Category Distribution

The dashboard represents the distribution of skills across categories such as **Machine Learning, DevOps, Programming, Database, Analytics, Backend, Other, and Cloud**. This provides an overview of the major skill domains represented in the dataset.

18.6 Salary Analysis

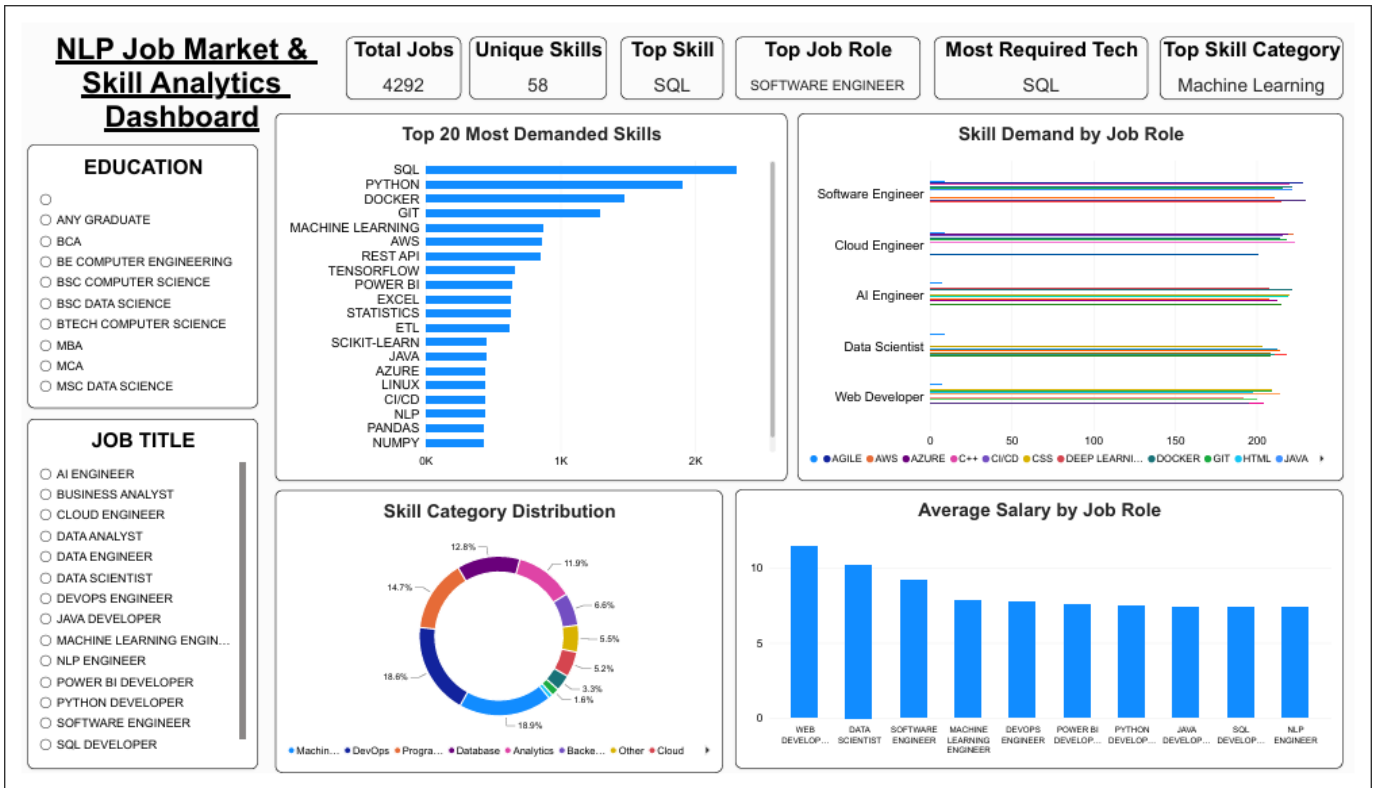
The dashboard also includes **Average Salary by Job Role**, allowing comparison of average salary values across roles such as Web Developer, Data Scientist, Software Engineer, Machine Learning Engineer, DevOps Engineer, Power BI Developer, Python Developer, Java Developer, SQL Developer, and NLP Engineer.

18.7 Dashboard Filters

Interactive filters are provided for **Education** and **Job Title**, allowing users to analyze the dashboard based on selected educational qualifications and job roles.

18.8 Visualization Summary

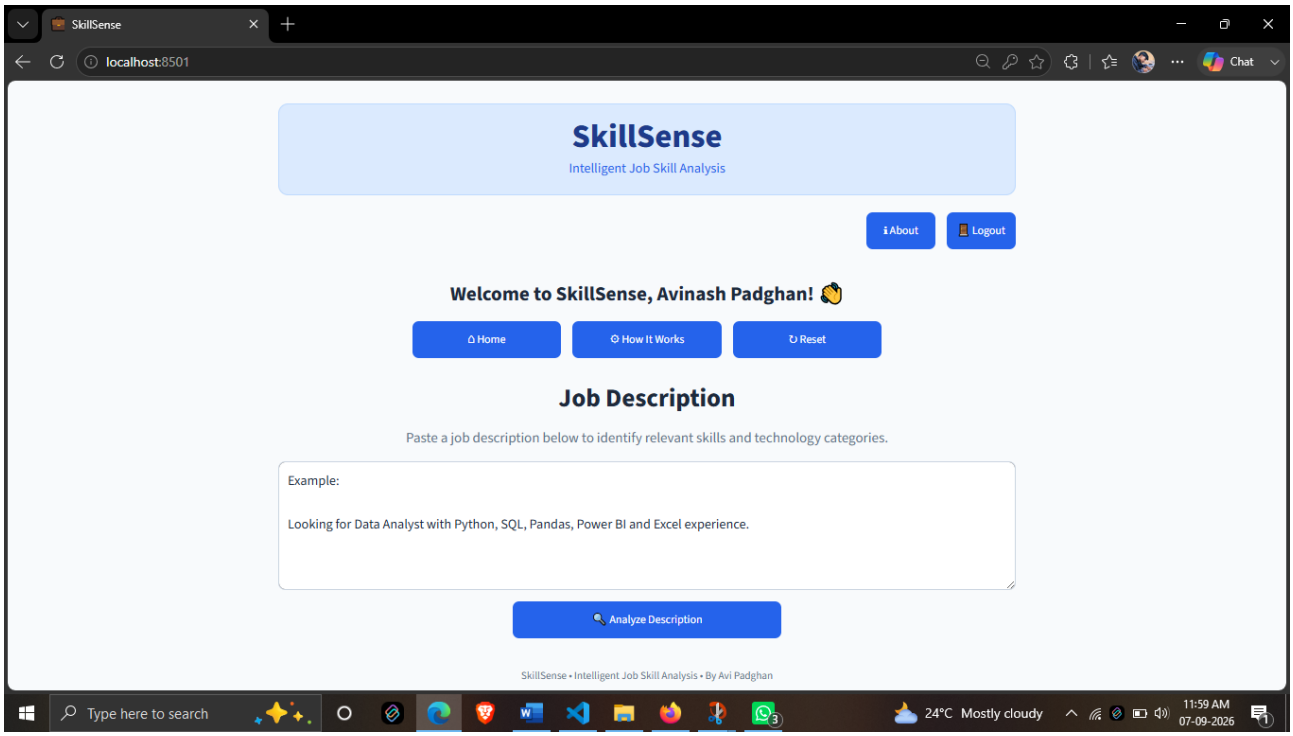
The Power BI dashboard converts the processed job and skill data into interactive visual insights. It helps users understand **skill demand, job-role requirements, skill categories, and salary patterns** in a single interface.


NLP Job Market & Skill Analytics Dashboard
Total Jobs
4292
Unique Skills
58
Top Skill
SQL
Top Job Role
SOFTWARE ENGINEER
Most Required Tech
SQL
Top Skill Category
Machine Learning
EDUCATION
ANY GRADUATE
BCA
BE COMPUTER ENGINEERING
BSC COMPUTER SCIENCE
BSC DATA SCIENCE
BTECH COMPUTER SCIENCE
MBA
MCA
MSC DATA SCIENCE
JOB TITLE
AI ENGINEER
BUSINESS ANALYST
CLOUD ENGINEER
DATA ANALYST
DATA ENGINEER
DATA SCIENTIST
DEVOPS ENGINEER
JAVA DEVELOPER
MACHINE LEARNING ENGIN...
NLP ENGINEER
POWER BI DEVELOPER
PYTHON DEVELOPER
SOFTWARE ENGINEER
SQL DEVELOPER
Top 20 Most Demanded Skills
SQL
PYTHON
DOCKER
GIT
MACHINE LEARNING
AWS
REST API
TENSORFLOW
POWER BI
EXCEL
STATISTICS
ETL
SCIKIT-LEARN
JAVA
AZURE
LINUX
CI/CD
NLP
PANDAS
NUMPY
0K
1K
2K
Skill Demand by Job Role
Software Engineer
Cloud Engineer
AI Engineer
Data Scientist
Web Developer
0
50
100
150
200
AGILE AWS AZURE C++ CI/CD CSS DEEP LEARNI... DOCKER GIT HTML JAVA
Skill Category Distribution
12.8%
11.9%
14.7%
6.6%
5.5%
5.2%
3.3%
1.8%
18.6%
18.9%
Machin... DevOps Progra... Database Analytics Backe... Other Cloud
Average Salary by Job Role
10
5
0
WEB DEVELOP...
DATA SCIENTIST
SOFTWARE ENGINEER
MACHINE LEARNING ENGINEER
DEVOPS ENGINEER
POWER BI DEVELOP...
PYTHON DEVELOP...
JAVA DEVELOP...
SQL DEVELOP...
NLP ENGINEER

19. APPLICATION / API DEVELOPMENT

19.1 Application Interface

The SkillSense application provides a user-friendly web interface developed using **Streamlit**. The interface allows users to enter a job description and submit it for automated skill analysis. The application connects the user interface with the trained NLP model and displays the extracted skills in a structured format.

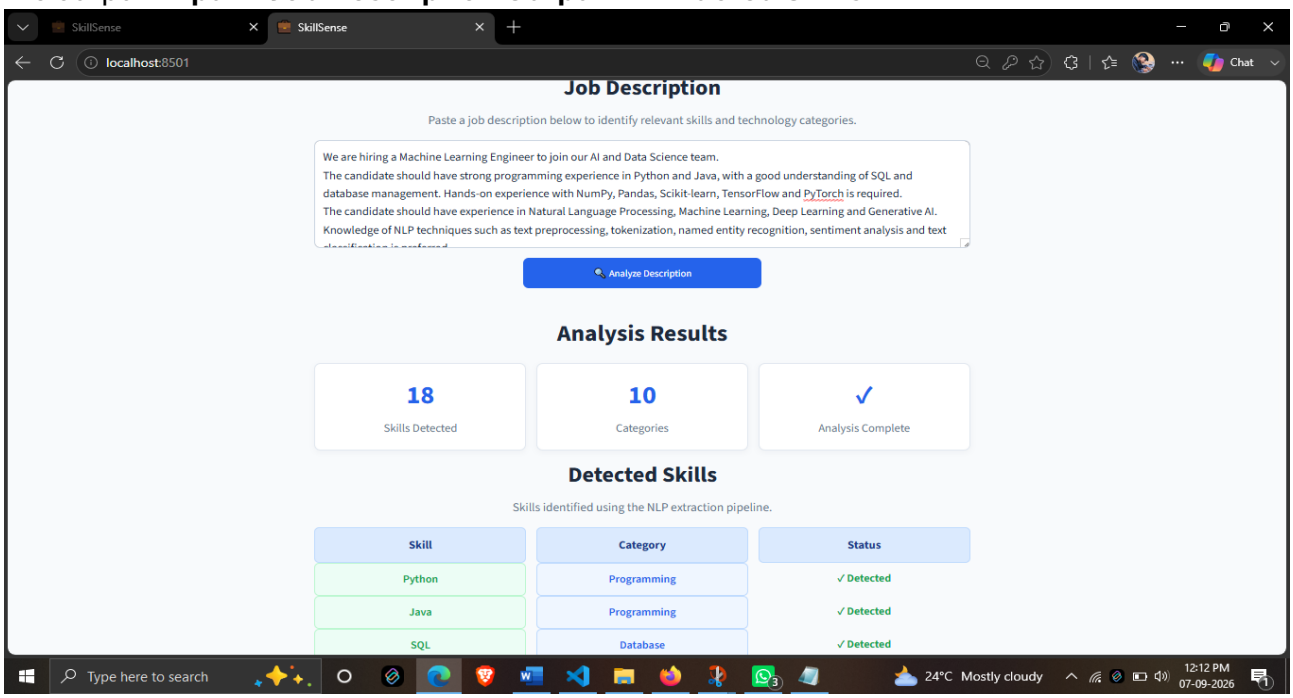


19.2 Input and Output

Input

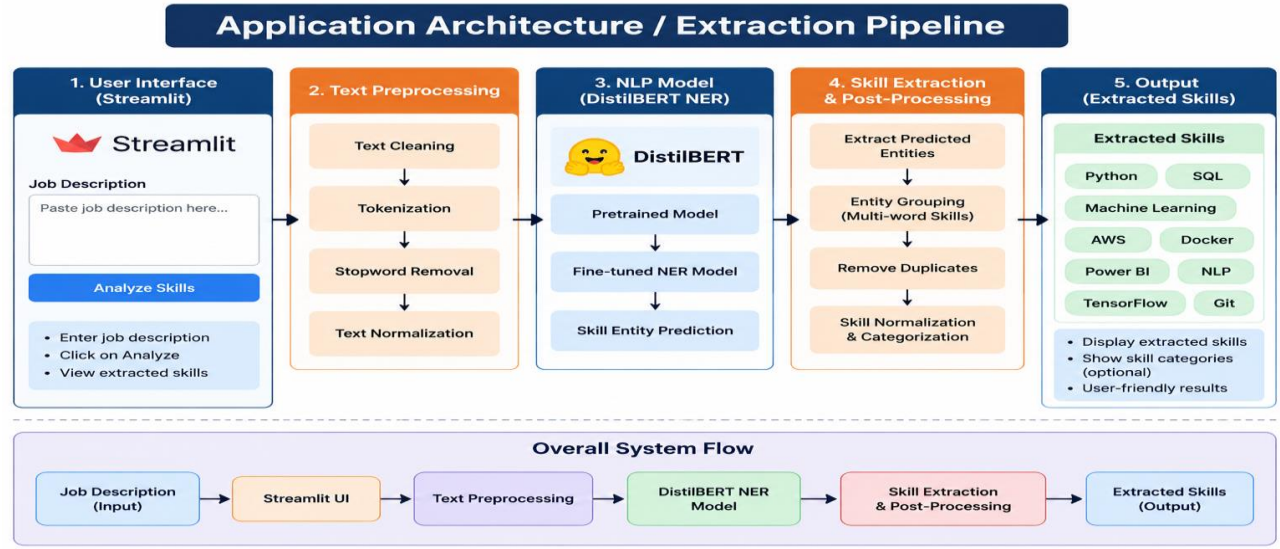
The primary input to the application is an **unstructured job description** containing information about job requirements, technologies, qualifications, and skills.

Output The system processes the input job description and provides the **identified skills** as the output .**Input → Job Description Output → Extracted Skills**



19.3 Skill Extraction Pipeline

The application follows the following extraction pipeline:



This pipeline connects the frontend interface with the NLP processing and skill extraction components.

19.4 Application Architecture

The application can be divided into three major components:

1. User Interface

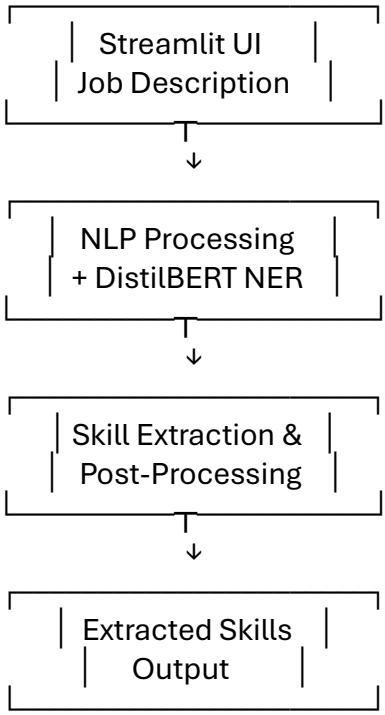
Streamlit provides the interface for entering job descriptions and viewing results.

2. NLP Processing Layer

The input text is processed and passed to the fine-tuned **DistilBERT-based NER model**.

3. Output Layer

The predicted skill entities are processed and displayed as the final skill list.



19.5 Application Result

The developed application provides an interactive way to analyze job descriptions without requiring the user to directly interact with the underlying Python or NLP code. It demonstrates the integration of the trained skill extraction model into a practical web-based application.

20. TESTING & RESULTS

20.1 Testing

The SkillSense application was tested using different job-description inputs to verify the functionality of the NLP-based skill extraction pipeline. Testing was performed to ensure that the application accepts input correctly, processes the text, extracts relevant skills, and displays the output properly.

20.2 Test Cases

Test Case	Input / Condition	Expected Result	Status
TC01	Valid job description	Skills should be extracted	Pass
TC02	Job description with multiple skills	Multiple skills should be identified	Pass
TC03	Multi-word skills	Complete skill phrases should be extracted	Pass
TC04	Different skill variations	Relevant skill variations should be handled	Pass
TC05	Empty/invalid input	Application should handle input appropriately	Pass

20.3 Sample Output

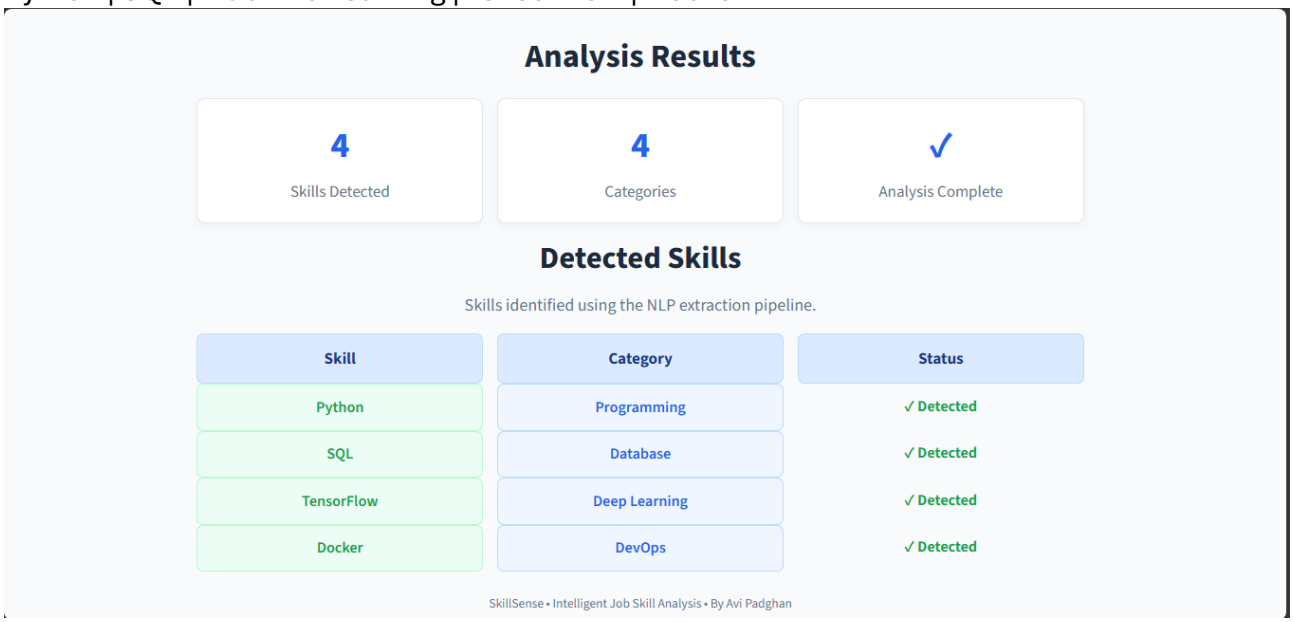
For a sample job description containing technical requirements such as **Python, SQL, Machine Learning, TensorFlow, and Docker**, the system processes the text and generates a structured list of extracted skills.

Sample Input:

Looking for a developer with experience in Python, SQL, Machine Learning, TensorFlow and Docker.

Expected Output:

Python | SQL | Machine Learning | TensorFlow | Docker



20.4 Validation

The extracted results were validated by comparing the system output with the expected skills present in the input job descriptions. Model performance was also evaluated using standard metrics such as **Precision, Recall, F1-Score, and Accuracy**.

For the implemented **TF-IDF + Logistic Regression** classification approach, the obtained accuracy was **21.43%**, with Precision of **2%**, Recall of **10%**, and F1-Score of **4%**.

20.5 Final Results

The final system successfully demonstrates an end-to-end job-skill analysis workflow:

Job Description → NLP Processing → Skill Extraction → Normalization → Structured Output

The developed application provides a practical interface for extracting and analyzing skills from job descriptions. The project also demonstrates the use of NLP, transformer-based techniques, machine learning, visualization, and dashboard-based analytics for job-market skill analysis.

Metric	Result
Accuracy	21.43%
Precision	2%
Recall	10%
F1-Score	4%

21. CHALLENGES & LIMITATIONS

21.1 Problems Faced

During the development of SkillSense, several challenges were encountered while working with job-description data and NLP models.

- Job descriptions contain different formats and writing styles.
- The same skill may appear using different names or abbreviations.
- Multi-word skills can be difficult to identify correctly.
- Some text may contain unnecessary or irrelevant information.
- Integrating the NLP model with the Streamlit application required proper input-output handling.

21.2 Model Limitations

The skill extraction model has some limitations:

- The model may produce **false positives** by identifying non-skill terms as entities.
- Some actual skills may be missed, resulting in **false negatives**.
- Multi-word skills may sometimes be extracted as partial entities.
- Model performance depends on the quality and coverage of the training data.
- New or uncommon technologies may not always be recognized correctly.

21.3 Dataset Limitations

The dataset also presents certain limitations:

- The dataset contains a limited number of job records compared with the complete job market.
- Skill representation may vary across different job descriptions.
- Some job roles or technologies may be underrepresented.
- Dataset quality directly affects the training and evaluation results.
- The dataset may not cover every newly emerging skill or technology.

21.4 Overall Limitation

The SkillSense system provides automated skill analysis, but its output should be considered **supporting information rather than a replacement for human judgment**. Improving the dataset, skill taxonomy, normalization rules, and model training can further improve the system's performance.

22. CONCLUSION

The **SkillSense – Intelligent Job Skill Analysis** project successfully demonstrates the application of **Natural Language Processing and Machine Learning techniques** for extracting relevant skills from job descriptions.

The project covers the complete workflow from **dataset analysis and NLP preprocessing to skill extraction, normalization, model evaluation, web application development, and dashboard visualization**. A **DistilBERT-based NER approach** was used for domain-specific skill identification, while rule-based and TF-IDF-based approaches provided additional methods for text and skill analysis.

The developed **Streamlit application** provides a user-friendly interface for submitting job descriptions and obtaining structured skill information. The **Power BI dashboard** further presents job-market insights such as top skills, skill demand by job role, skill categories, and salary analysis.

Key Achievements

- Developed an NLP-based job-skill extraction system.
- Analyzed **4,200+ job records** during the project workflow.
- Implemented TF-IDF-based text representation.
- Developed a DistilBERT-based NER approach for skill extraction.
- Applied skill normalization and categorization concepts.
- Evaluated the machine learning approach using standard metrics.
- Developed an interactive **Streamlit web application**.
- Created a **Power BI skill analytics dashboard**.
- Identified practical applications in recruitment, skill analysis, and career planning.

Overall, the project demonstrates how NLP, transformer models, machine learning, and data visualization can be combined to convert unstructured job descriptions into **structured and meaningful skill insights**.

23. FUTURE SCOPE

The SkillSense project can be further extended to provide more advanced recruitment and career-oriented features. The current skill extraction system can serve as a foundation for building intelligent job-matching and recommendation solutions.

23.1 Resume–Job Matching

The system can be extended to compare the skills extracted from a **resume** with the skills required in a job description. A matching score can be generated to indicate how closely the candidate profile matches the job requirements.

23.2 Candidate Ranking

Multiple candidates can be ranked based on their skills, experience, qualifications, and similarity to the required job profile. This can help recruiters shortlist suitable candidates more efficiently.

23.3 Skill-Gap Analysis

The system can identify the skills required by a job but missing from a candidate's resume. This can help candidates understand their **skill gaps** and identify areas for improvement.

23.4 Recommendation System

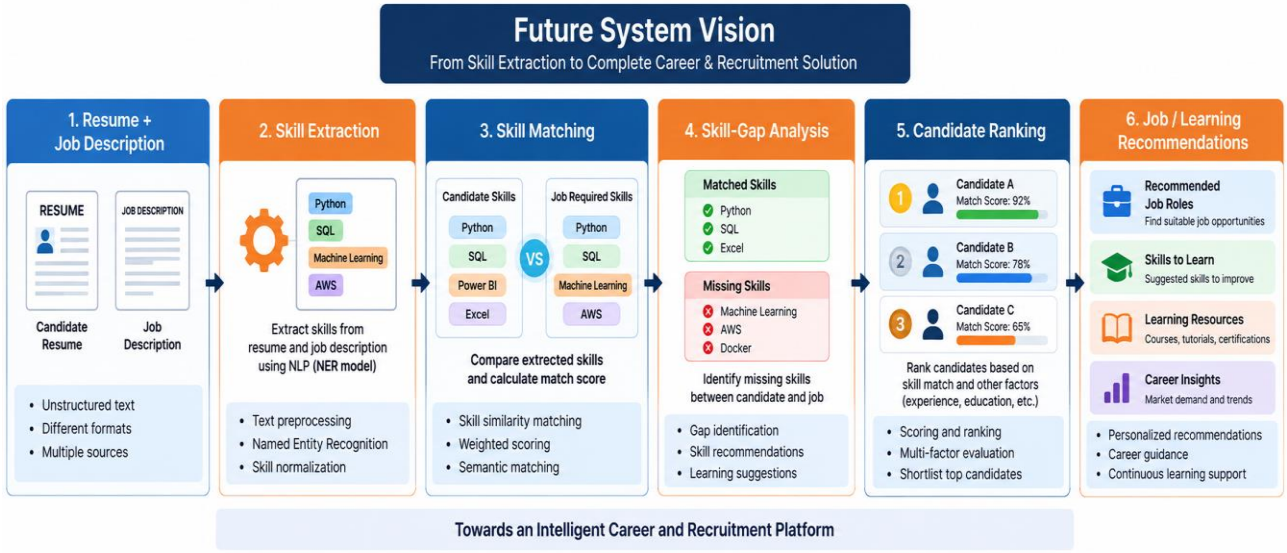
A recommendation system can be developed to suggest:

- Suitable job roles
- Relevant skills to learn
- Learning resources
- Career opportunities

Recommendations can be generated based on the candidate's existing skills and target job requirements.

23.5 Future System Vision

The future version of SkillSense can evolve from a **skill extraction system** into a complete intelligent career and recruitment platform:



These enhancements can make SkillSense more useful for **recruiters, job seekers, students, and organizations**.

24. INTERNSHIP LEARNING OUTCOMES

During the internship, practical knowledge and hands-on experience were gained in **technical development, data analysis, NLP, machine learning, and project development**. The project provided an opportunity to apply theoretical concepts to a real-world job-skill extraction problem.

24.1 Technical Skills

- Python programming for NLP and Machine Learning.
- Working with **Pandas, NumPy, NLTK, spaCy, Scikit-learn, and Transformers**.
- Data preprocessing and text cleaning.
- Working with **DistilBERT** and NER models.
- Streamlit application development.
- Power BI dashboard development.
- Git and GitHub for project management.

24.2 Analytical Skills

- Performing Exploratory Data Analysis (EDA).
- Analyzing job-market and skill-demand data.
- Identifying patterns across job roles and locations.
- Creating visualizations using **Matplotlib and Seaborn**.
- Interpreting model evaluation results.
- Drawing meaningful insights from extracted skills.

24.3 NLP Skills

- Text preprocessing and tokenization.
- Understanding stopwords, stemming, and lemmatization.
- TF-IDF and N-Gram analysis.
- Named Entity Recognition (NER).
- Skill extraction from unstructured job descriptions.
- Skill normalization, alias matching, and categorization.
- Working with transformer-based NLP models.

24.4 Machine Learning Skills

- Preparing training and testing data.
- Feature engineering using TF-IDF.
- Implementing **Logistic Regression**.
- Understanding model training and prediction.
- Evaluating models using **Accuracy, Precision, Recall, and F1-Score**.
- Performing error analysis and understanding model limitations.

24.5 Project Development Skills

- Developing an end-to-end NLP project.
- Integrating the NLP model with a **Streamlit web application**.
- Creating an interactive **Power BI dashboard**.
- Connecting data processing, model, application, and visualization components.
- Debugging and solving implementation issues.
- Preparing technical documentation and project reports.

24.6 Overall Learning Outcome

The internship provided practical experience in applying **NLP, Machine Learning, data analysis, and application development** to a real-world problem. It improved technical knowledge, analytical thinking, problem-solving ability, and understanding of the complete **data → model → application → dashboard** development lifecycle.

Internship Learning Outcomes

Skills Gained | Practical Experience | Real-World Application

Learn | Build | Grow

Technical Skills

- Python programming for NLP and ML
- Working with libraries (NumPy, Pandas, Scikit-learn, Transformers, SpaCy)
- Data preprocessing and text cleaning
- Model training, evaluation and hyperparameter tuning
- Version control using Git and GitHub
- Streamlit for web application development
- Power BI for data visualization

Analytical Skills

- Analyzing job market data and identifying trends
- Data exploration and visualization
- Understanding skill demand across different job roles
- Drawing insights from extracted skills
- Interpreting model evaluation results
- Making data-driven decisions during development

NLP Skills

- Text preprocessing (tokenization, stopword removal, lemmatization)
- Named Entity Recognition (NER) for skill extraction
- Working with transformer models (DistilBERT)
- Handling unstructured text data (job descriptions)
- Skill normalization and alias matching
- Understanding challenges in real-world NLP data

Machine Learning Skills

- Implementing TF-IDF for text representation
- Building and evaluating classification models (Logistic Regression)
- Working with DistilBERT for domain-specific NER
- Model performance analysis (Precision, Recall, F1-Score, Accuracy)
- Understanding model limitations and error analysis
- Applying ML techniques to real-world problems

Project Development Skills

- End-to-end project development (data → model → application → dashboard)
- Problem solving and troubleshooting
- Integrating NLP model with Streamlit application
- Designing a user-friendly interface
- Creating interactive Power BI dashboard
- Documentation and reporting
- Time management and independent learning
- Experience in working on a real-world project

Overall Outcome

The internship provided hands-on experience in applying NLP, Machine Learning, data analysis, and full-stack project development skills to solve a real-world problem. It helped in building technical knowledge, analytical thinking, and practical implementation skills, preparing for future opportunities in AI and data-driven domains.

- Applied academic knowledge in a real project
- Gained industry-relevant skills
- Improved problem-solving and research abilities
- Built confidence in developing AI/ML solutions

25. REFERENCES

25.1 Dataset Source

1. **ChatGPT (OpenAI)** – Used to assist in generating and structuring the synthetic Job Description dataset used in the SkillSense project.
   OpenAI / ChatGPT

25.2 Python Libraries & Documentation

2. **Python Documentation** – Python programming and standard libraries.
   Python Documentation
3. **Pandas Documentation** – Data manipulation and analysis.
   Pandas Documentation
4. **NumPy Documentation** – Numerical computing and array operations.
   NumPy Documentation
5. **NLTK Documentation** – Natural Language Processing and text preprocessing.
   NLTK Documentation
6. **spaCy Documentation** – NLP processing and linguistic analysis.
   spaCy Documentation
7. **Scikit-learn Documentation** – Machine learning, TF-IDF, Logistic Regression and model evaluation.
   Scikit-learn Documentation
8. **Hugging Face Transformers Documentation** – Transformer models, tokenization, training and inference.
   Hugging Face Transformers Documentation
9. **Matplotlib Documentation** – Data visualization and plotting.
   Matplotlib Documentation
10. **Seaborn Documentation** – Statistical data visualization.
    Seaborn Documentation

25.3 Tools & Platforms

11. **Streamlit Documentation** – Interactive web application development.
    Streamlit Documentation
12. **Microsoft Power BI Documentation** – Dashboard and data visualization.
    Power BI Documentation
13. **Project Jupyter Documentation** – Interactive notebooks and data analysis.
    Jupyter Documentation
14. **GitHub Documentation** – Version control and project development resources.
    GitHub Documentation

25.4 Research Papers

15. **Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K.**
    BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
    BERT Research Paper
16. **Sanh, V., Debut, L., Chaumond, J., & Wolf, T.**
    DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.
    DistilBERT Research Paper
17. **Vaswani, A., et al.**
    Attention Is All You Need.
    Transformer Research Paper